Soar 9.6 on the Common Model of Cognition

A Lean 4 specification of Soar built on the CMC specification, checked against the Soar 9.6.5 release

Status report · November 1, 2026, 1:10 PM EDT · Prepared for the Center for Integrated Cognition

1. Summary

Soar 9.6 has a formal, executable specification in Lean 4 that is built on top of the Lean specification of the Common Model of Cognition (CMC). The CMC files define what any conforming architecture must provide and satisfy. The Soar files define Soar's working memory, production language, matcher, preference semantics and decision cycle precisely enough to run, and Soar is constructed as an instance of the CMC specification with its proof obligations discharged.

The specification runs ordinary Soar programs. A command-line tool, soar-lean, reads a .soar file and runs it on the formal model. That makes it possible to check the specification run by run against the Soar 9.6.5 release: the same program is given to both, and the decision traces and final working memories are compared. They agree exactly on all 543 hand-written and generated programs and on all 13 agents from the Soar repository's own test suite, including blocks world, look-ahead search, eight puzzle, Towers of Hanoi, water jug and multi-column arithmetic.

The specification covers reinforcement learning (Section 6). With learning on and exploration enabled, the model and Soar 9.6.5 make the same decisions and learn the same values, to printed precision, on all 140 reinforcement-learning programs tested, including hierarchical tasks where learning operators fall into operator no-change impasses, Q-learning, eligibility traces, on- and off-policy GQ(λ), every decay mode including delta-bar-delta meta-learning, and template rules. It also covers exploration: every indifferent-selection policy, with Soar's random-number generator reproduced bit for bit, so that with the same seed the model makes the same random choices as Soar (17 purpose-written programs and 66 of the 66 reinforcement-learning programs match without replaying Soar's choices).

It covers semantic memory (Section 7): storage, cue-based and non-cue-based retrieval with all their modifiers, recency, frequency, base-level and spreading activation, smem --add, --remove and --query, and the long-term-identifier tests in conditions. The model matches Soar 9.6.5 on 759 of 760 semantic-memory programs, among them 37 of the 38 semantic-memory functional tests in the Soar repository, and on all 150 further random programs with spreading activation. The remaining functional test, which uses spreading with edge updating and working-memory activation as its source, matches in decisions and in the store with every spread activation, and differs in four working-memory activations where Soar reads freed memory (Section 12).

It covers episodic memory (Section 8): recording an episode every decision cycle or on output, cue-based retrieval with negative cues, before, after, prohibit and graph matching, retrieval by episode number, next and previous, and all the result meta-data. Its store is modeled on Soar 9.6.5's incremental encoder and interval search, because which episode a query returns can depend on how stored structures are identified over time. The model matches Soar 9.6.5 on 671 episodic-memory programs, comparing every stored episode as well as decisions and working memory, including all 44 episodic-memory functional tests in the Soar repository that can run without the spatial-visual system, among them those with working-memory activation. Both stores can be kept in files between agents, as in Soar; all 68 two-run programs match, in which a second agent continues from the file the first one left

(tests/dbfile, among them the repository's Hamilton store and retrieve tests with their file settings restored).

It covers working-memory activation (Section 9): decay elements for o-supported elements, the references Soar counts from tested elements, preferences and the o-sets of i-supported elements, the initial boost of new elements, Petrov's approximation, forgetting in all three modes, and the weighting of episodic-memory cues by activation. The model matches Soar 9.6.5 on 324 of 325 programs, comparing every element's printed activation, including the 4 working-memory-activation tests in the Soar repository; the other differs in a graph-match mapping that Soar takes in memory-address order (Section 12).

It covers chunking (Section 10), as Soar 9.6.5's explanation-based chunking: identities for the variables of every instantiation that fires in a substate, dependency analysis through those instantiations (with each condition traced, as in Soar, to the preference that supported its element when the instantiation fired), joining and literalization of identities, singletons (automatic and declared), propagation of constraints with the tests where Soar's condition reorderer puts them, negated conditions, merging, operator selection knowledge, rule repair, learning from semantic- and episodic-memory retrievals, add-ltm-links, deep-copy, bottom-only learning, and the formation, naming and instantiation of learned rules, including chunks learned from chunks and from justifications. Learned rules are compared with the rules Soar 9.6.5 learns, up to renaming of variables and order of conditions. The model learns the same rules and makes the same decisions with the same final working memory on all 86 chunking unit-test agents in the Soar repository and on 17 variants of them with other settings. On the repository's look-ahead and subgoaling demonstration agents, with Soar's default settings, all 13 agents match in every respect over 25 decision cycles.

Formalizing and testing together exposed 43 places where the Soar 9.6 manual is wrong, silent or ambiguous about what Soar 9.6.5 does (Section 11), twenty-nine defects across eight modules of the Soar 9.6.5 kernel, six of them severe (a crash, reads of freed or out-of-bounds memory, a NULL dereference, and memory growth that exhausts a machine), each located in the source with a recommended fix in a companion document (Section 12), and five findings for the CMC (Section 13), including how semantic and episodic memory should map onto the CMC's declarative module, where a preferred option is recorded and deferred.

Measure	Value
Lean code	About 21,800 lines (Lean 4.35.0, core library only) on top of the existing CMC files
Proved theorems	563 (9 for the expanded CMC backbone, 554 about Soar); no unproved steps; only Lean's standard axioms, also in the compiled program, whose hash-table lookup is proved equal to the matcher it stands for (Section 4)
Programs checked against Soar 9.6.5	543 hand-written and generated programs (543 identical) and 13 of 13 test-suite agents, identical in decisions and final working memory, among them programs that take input from an environment; 140 reinforcement-learning programs identical in decisions and learned values; 760 semantic-memory programs (759 identical in decisions, working memory and the contents and activations of semantic memory; the other differs only in four working-memory activations) and 150 with spreading activation (150 identical; all identical in the store); 325 working-memory-

Measure	Value
	activation programs (324 identical, with every printed activation compared); 671 episodic-memory programs identical in decisions, working memory and every stored episode; and, built on November 1 to look for differences in the order of matches, 300 programs with justifications, 450 with batches of working-memory changes (semantic and episodic queries, retrieval and depth) and 12 with nested negations in chunking, all identical
Learned rules checked against Soar 9.6.5	86 of 86 chunking unit-test agents, 17 variants and 5 programs learning through memory retrievals identical in learned rules, decisions and final working memory; 13 of 13 demonstration agents identical over 25 decision cycles
Manual issues found	43 (Section 11)
Kernel defects and anomalies found	29 (Section 12)

With spreading activation completed on October 31, every feature of Soar 9.6.5 outside the spatial-visual system and the exclusions decided on October 29 is formalized. Section 2 lists the known differences that remain (orders the model does not reproduce and Soar's undefined behavior) and the work that remains on proofs.

2. Status: what is done and what remains

The plan was to formalize the core of Soar first, then add the learning mechanisms and long-term declarative memories as separate extensions, and to leave out the spatial-visual system. The core is complete and runs real Soar programs. Reinforcement learning, semantic memory, episodic memory, working-memory activation and chunking are built on top of it.

Component	Status	Notes
Working memory, symbols, production language	Done	Integers, floating point, symbolic constants; variables, predicates, disjunctions, conjunctive tests, negated conditions and negated conjunctions (nested to any depth), structured values, acceptable-preference tests, long-term-identifier tests
Right-hand side	Done	Working-memory changes, all eleven preferences, Soar 9.6.5's whole function library except from-st-xml (Section 5); accept gives no value, as Soar does at end of input: + - * / div mod abs sqrt sin cos atan2 int float (both also of strings, as strtoll and strtod read them) min max round-off round-off-heading compute-heading compute-range extrapolate-x-position extrapolate-y-position haversine compute-dice-probability select-point-closest-to-vector size sum product set-count set-sum set-multiply set-min set-max set-range set-mean set-stdev rand-int rand-float string strlen trim capitalize-symbol ifeq make-constant-symbol dc timestamp @ concat deep-copy crlf; link-stm-to-ltm; write, halt, log, trace, count and wait as actions,

Component	Status	Notes
		interrupt (at the end of the phase), cmd (epmem settings), force-learn, dont-learn
Matcher	Done	Soundness proved; Soar-style condition ordering
Preference semantics (manual §2.4.2)	Done	Eight-step resolution and impasse classification; properties proved
Preference memory and timetags	Done	Preferences kept per slot in Soar 9.6.5's order, each with its instantiation, support, level and goal; working-memory elements made from them in the working-memory phase; o-supported rejects first; clones of result preferences for justifications and learned rules; timetags on elements, with matches identified by the timetags of the elements they matched; identifier levels with Soar's promotion and demotion
Decision cycle	Done	Input, proposal, decision, application, output; Soar 9.6.5's outer and inner elaboration loops (highest active state, IE or PE chosen per state and resumed after an interruption, the waterfall rule, retractions at the highest active state only); consistency checks at minor quiescence and on changes of level; decision from the highest state whose operator slot changed; operator removal when its acceptable preference is gone; i- and o-support as the Rete classifies a match, including operator augmentations; max-elaborations; the order in which Soar's Rete takes new matches and retractions, and Soar's order of making elements (Section 5)
Impasses and substates	Done	All five impasse types, substate structures including the semantic-memory, episodic-memory and reward links, stacking, removal
Results, justifications, goal dependency set	Done	Backtracing to the level a result attaches to; GDS recording and regeneration
Reading .soar files; soar-lean executable	Done	Attribute paths, multi-valued shorthand, quoted constants with escapes; gp; negations nested to any depth, also inside structured values; preferences on attributes other than ^operator as Soar treats them; the long-term-identifier tests @, !@, @+ and @-; :template rules; source, pushd, popd and cd; the soar settings max-elaborations, max-goal-depth, wait-snc and keep-all-top-oprefs, and every rl --set parameter of the formalized features
Input from an environment	Done	The input phase places the elements an environment gives (an input function of the decision count and the output link); soar-lean reads a script of input-link changes by decision cycle, recorded from Soar 9.6.5 through SML for reactive environments (Section 5, B.9)
Expanded CMC backbone and Soar as	Done	CMC theorems re-proved for the expanded procedural

Component	Status	Notes
an instance		module
Comparison with Soar 9.6.5	Done	543 of 543 hand-written and generated programs and 13 of 13 test-suite agents matching
Reinforcement learning	Done	Sarsa and Q-learning, on- and off-policy GQ(λ), with the expected current and future reward split; eligibility traces (λ > 0, with the tolerance); the decay modes normal, exponential, logarithmic and delta-bar-delta (meta-learning); temporal extension (gaps), temporal-discount, hierarchical discounting; updates on state retraction and halt; :template rules (Section 6)
Exploration and random numbers	Done	Soar 9.6.5's random-number generator (MT19937, srand), bit for bit; indifferent selection by first, last, softmax, epsilon-greedy and boltzmann; numeric-indifferent-mode sum and avg; auto-reduce of epsilon and temperature, exponential and linear; rand-int and rand-float; Soar's order of the candidates (Section 6)
Semantic memory	Done	Spreading activation (Section 7); store, store-new, cue-based queries with any-value, constant and LTI cue elements, prohibit, neg-query, math-query, depth and link-to-ltm, non-cue-based retrieval, status elements, recency, frequency and base-level activation (stable, naive and incremental policies), the separate ordering for LTIs with thresh or more augmentations, smem --add with aliases, smem --remove and smem --query, the long-term-identifier tests and the @ function, link-stm-to-ltm; the mapping onto the CMC is deferred, with a preferred option recorded (Section 13)
Episodic memory	Done	Recording with the dc and output triggers, force and exclusions, in the output or selection phase; cue-based retrieval with neg-query, before, after, prohibit, graph match and the mapping; retrieve, next, previous; all meta-data; episodes in which an identifier changes the LTI it stands for; merge and lazy-commit accepted (no observable effect in 9.6.5); the mapping onto the CMC is deferred, with a preferred option recorded (Section 13)
Working-memory activation	Done	Decay elements, references from tested elements, preferences and o-sets, initial boost, Petrov's approximation, forgetting (naive, bsearch, on; forget-wme lti; fake-forgetting), episodic-memory cue weights with balance below 1 (Section 9); spreading-wma-source (Section 7)
Chunking, phase 1	Done	Explanation-based chunking as in Soar 9.6.5 (Section 10): identities, dependency analysis, singletons, constraints, $ literalization, negations, merging, naming, duplicates, max-chunks and max-dupes, bottom-up learning, chunk always|never|only|except;

Component	Status	Notes
		learned rules kept in procedural module state (decided, Section 13)
Chunking, phase 2	Done	Operator selection knowledge (on by default in 9.6.5), rule repair, learning from semantic-memory retrievals, deep-copy, RL proposals in chunks, chunk singleton, bottom-only, traces fixed when an instantiation fires, Soar's condition reorderer for the placement of relational tests
Chunking, phase 3	Done	Learning from episodic-memory retrievals; add-ltm-links. Operator selection knowledge from rules that match only the top state arises only when a substate is returned as a result, and gives Soar a rule with literal identifiers; it is recorded as a finding, not modeled (Sections 11 and 12)
Chunking, phase 4	To do	Proofs about the learning computation; the CMC.Proc iteration (Section 13)
Further proofs	In part	Done: matcher completeness for positive conditions, negations and negated conjunctions with one level of inner negations; that reordering the positive patterns preserves matches; that the join order returns the rule's own conditions, exactly reordered when no relational test between variables has to move; at most one selected operator per state over whole runs (Section 4). Also done: completeness for negated conjunctions nested to any depth (tree_complete, tree_clear); that the join order keeps the matches of the positive patterns when its moved relational tests compare values of one kind (joinOrder_matches; between values of different types they are not preserved, in Soar too: kernel defect 6); that reach, the link of working memory to a state, is correct (mem_reach_iff). To do: the same for negations and for Soar's reorderer
Spatial-visual system	Out of scope	As agreed
The other exclusions	Out of scope	Decided October 29, 2026; listed below

What remains for a complete specification of Soar without the spatial-visual system

Every feature of Soar 9.6.5 outside the spatial-visual system and the exclusions below is now formalized: the decision cycle, reinforcement learning, semantic memory with base-level and spreading activation, episodic memory, working-memory activation and chunking. What remains is not features but the following.

Area	Remaining
Back in scope and done (October 31)	Semantic-memory databases on disk (smem --set database file, path and append, with --backup, --clear and --init); the episodic-memory database on disk; rules made

Area	Remaining
	from RL templates, also while chunking.
Proofs	Proofs about the learning computation (chunking phase 4); that Soar's reorderer, and negations, keep matches when relational tests move.
Orders not reproduced	The memory-address order in which Soar's episodic graph match tries candidate mappings (Section 12). The Rete's order is modeled (Section 5), with the limits listed with the ad hoc places (Section 10).
Soar's undefined behavior	Reads of freed preferences and elements in working-memory activation, and of constants as identifiers in spreading's edge updating; the model takes the evident intent and reports the reads (Section 12).

Remaining differences from Soar 9.6.5. All 13 chunking demonstration agents match (Section 10). The remaining differences are: through the memory-address order of the episodic graph match, one episodic program with working-memory activation and one random episodic program (f229) that matches in some runs of Soar 9.6.5 and not in others; through Soar's undefined reads, the semantic-memory functional test AlphabetAgentAllOn (four working-memory activations) and 9 of 30 programs with edge updating. doc/DIFFERENCES.md records how each was diagnosed. Every program of the sets also run without Soar's choices replayed (Sections 5 and 6) matches that way too.

What the model establishes. Agreement with Soar 9.6.5 on the covered features is established by differential testing on the programs listed in Sections 5 to 10, not by proof. The proved theorems are properties of the model (invariants, and what each module can and cannot change); they are not a proof that the model is equivalent to Soar 9.6.5.

Excluded by decision (October 29, 2026). The explainer will not be formalized. Working-memory activation and forgetting, with the episodic-memory balance parameter below 1, were excluded on October 29 and brought back into scope on October 30; semantic-memory databases on disk, episodic-memory databases on disk and reinforcement-learning templates in learned rules were excluded on October 29, brought back into scope on October 31 and formalized the same day. With the explainer and the spatial-visual system excluded, the model is a complete specification apart from the items in the table above.

3. How the Soar specification is built on the CMC specification

The work is organized in layers. Each layer imports the one below, and the lower layers are not modified.

Layer	Contents
CMC.Base	The 2017 Standard Model with the clarifications decided in October 2026 (unchanged in content; see Appendix A for the edits Lean 4.35.0 required)
CMC.Meta	The metacognition extension: process-data areas for every module, selection-failure reports, episodic recording (unchanged in content; see Appendix A)
CMC.Proc	New: the CMC with an expanded procedural module, required to cover Soar (below)
Soar core	Syntax, matcher, preferences, decision cycle, properties of the cycle, parser

Layer	Contents
Soar.Conform	Soar as an instance of CMC.Proc, with every proof obligation discharged

What had to change in the CMC

The CMC says procedural memory induces the selection of a single deliberate act per cycle. Soar has that, but also rules that fire automatically and in parallel before and after the selection. Four decisions settle how Soar fits.

Decision	Content
Automatic and deliberate processing	Rules are either retractable (i-support: elaboration, proposal, evaluation) or tied to the selected operator (o-support: persistent, destructive changes). The two kinds fire separately; the operator is selected at quiescence; selection is non-monotonic. The point of correspondence with the CMC is that a single operator is selected and applied per cycle, the CMC's single deliberate act.
Procedural module state	Preference memory, instantiation and support records, and the goal dependency set are internal state of the procedural module and cannot be tested by rules (acceptable preferences excepted), like the internal state of the perception and motor modules.
Architecture structures	Rules cannot overwrite architecture-created structures. The CMC's requirement is weakened from "procedural memory never writes these areas" to "never removes or replaces their content"; the stronger form is kept as a separate, optional property.
Substate structures	The initial structures of a substate are procedural process data, the CMC metacognition extension's report of a selection failure.

How Soar maps onto the CMC

Soar	CMC
Input-link elements	Perceptual buffer content
Rule-created elements under the output link	Motor command
State and substate structures created by the architecture	Procedural process data
All other working-memory elements, including operator selections	The region (working memory not associated with a module)
Input and proposal phases	Automatic procedural processing
Decision and application phases	Selection of the deliberate act and its application
Tie, conflict, state no-change, operator no-change	Selection failure: tie, conflict, no candidates, no progress
Constraint failure	Selection failure: conflict (the CMC has no separate category)

Because Soar is an instance, the CMC theorems apply to it. Among them: selection failures are always reported in procedural process data; architecture-written content is never overwritten by procedural processing; and declarative knowledge reaches behavior only through retrieval, re-proved with automatic processing and procedural module state included. Running Soar through the CMC interface gives exactly the same agent state as running it directly: `cmcStep_pst` proves that one cycle of the Soar instance is `St.cycle`, and `runCMC_pst` that running n of them is `St.runN` (Soar's run, without the reinforcement-learning update at a halt), for every agent.

The decision cycle in the model

Figure 1 shows one decision cycle as the model runs it (St.cycle in Soar/Cycle.lean): the five phases, the elaboration cycle that the proposal and application phases repeat, and the points at which reinforcement learning, semantic memory, episodic memory and chunking act. It follows the control structure of the Soar 9.6.5 source (do_preference_phase, do_working_memory_phase, decide_context_slots and the determine_highest_active_production_level functions), including the inner elaboration loop, in which the rules of lower states fire in the same elaboration cycle as those of the highest active state.

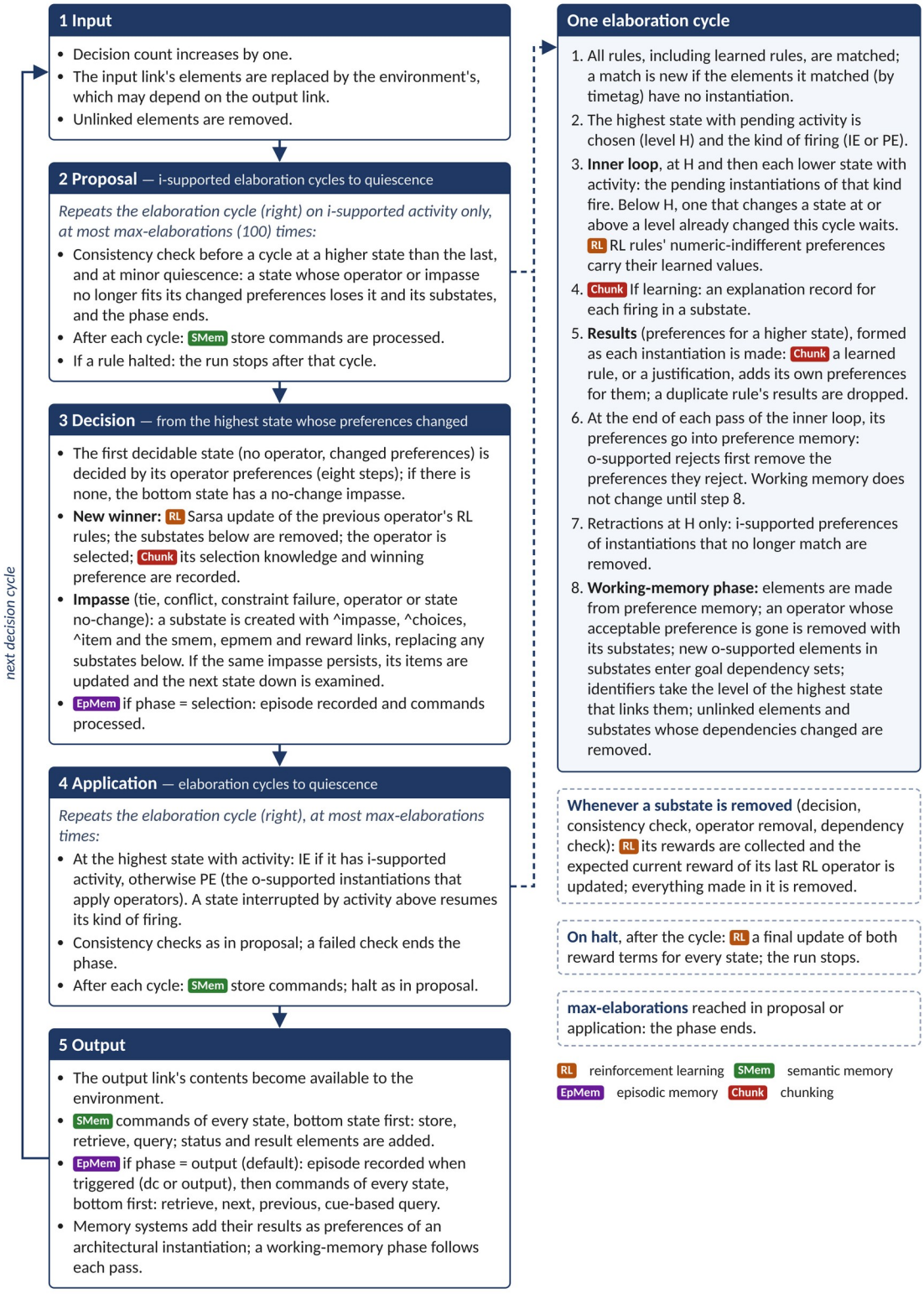


Figure 1. The decision cycle in the model. Proposal and application repeat the elaboration cycle on the right; the tags mark where each learning mechanism and memory acts.

4. What is proved about Soar

There are 563 theorems. The statements about the cycle are made over preference memory, the elaboration loops and the consistency checks of the model as described in Sections 3 and 5, and hold for whole proposal and application phases. The main results:

- **Matching is sound.** Every instantiation the matcher returns satisfies each positive condition, through the element recorded for it, under the instantiation's bindings, long-term-identifier tests included. The indexed matcher the program runs, which looks elements up by identifier, returns exactly the instantiations of the plain matcher, in the same order (matchCondsIx_eq), and its hash-table lookup is proved to return what filtering working memory returns (WmIndex.onId_eq_onIdTable), so the compiled program has no unproved step.
- **Matching is complete, and the order of patterns does not matter** (Soar/MatchComplete.lean). If, under some bindings, every positive condition matches an element of working memory and no negation or negated conjunction matches, the matcher finds those bindings, with exactly those elements (matchConds_complete_neg). The patterns must be in a safe order: a variable read by a relational test is bound before it, as Soar requires and both its reorderer and the model's join order arrange; a negation's own variables must be local to it. From soundness and completeness together, any reordering of the positive patterns into another safe order finds the same matches (matchPats_perm). Negated conjunctions nested to any depth are covered too: a negated conjunction is a tree, and tree_complete and tree_clear show that the matcher finds a match of it exactly when the specification says it holds, provided the variables of an inner negation that no enclosing node binds are local to it (the matcher's NegTree.holds is now a structural definition, so the proof can unfold it).
- **The join order keeps the conditions** (Soar/JoinOrder.lean). The order the matcher uses returns the rule's own conditions, reordered, up to where relational tests sit (joinOrder_strip), and exactly a permutation of them when the rule has no relational test between variables (joinOrder_perm); with matchPats_perm, the order then does not change the matches. When every remaining condition tests a variable not yet bound, the join order moves those tests, inverted, onto the conditions that bind the variables, as Soar's reorderer does. A moved test keeps its meaning between two numbers and between two identifiers (RelOp.holds_inv_num, RelOp.holds_inv_id), but not between values of different types, which Soar counts as less in either direction (RelOp.holds_inv_mixed; kernel defect 6): there, in Soar as in the model, the matches depend on where the test is placed. With a typing of the variables that gives each variable compared by a relational test the kind of number or identifier, moving tests does preserve the matches of the positive patterns (Soar/JoinOrderSem.lean: joinOrder_posSat, and for the matcher joinOrder_matches and joinOrder_matches_rev, which take the safe order of the patterns as a hypothesis); strings compared with strings, @ tests and negations are not covered.
- **Only candidates win.** A selected operator carries a require preference and no prohibit, or an acceptable preference and neither prohibit nor reject.
- **Impasses have the causes the manual gives.** Constraint failure only from several required values or a required and prohibited value; conflict only when every candidate is worse than another; tie only among at least two candidates that are not all mutually indifferent; a single unprohibited required value always wins.
- **Rules never change architecture structures; only the architecture removes them.** Over a whole proposal or application phase, and over each elaboration cycle and working-memory phase within it, no architecture-created element and no state is added, and every architecture-created element that is removed is an operator selection or belongs to a substate. A pass of the inner elaboration

loop — rule firings, results, justifications and learned rules — leaves every architecture-created element and every state unchanged, since working memory changes only in the working-memory phase. Such elements are removed only by the architecture: an operator whose acceptable preference is gone, a decision no longer consistent with the preferences, or a substate whose dependencies changed.

- **One selection per decision.** A decision phase adds at most one operator selection to working memory, the formal counterpart of the CMC's single deliberate act.
- **At most one selected operator per state, over whole runs** (Soar/Seriality.lean). From the initial state, after any number of decision cycles, two operator selections of the same state are the same element (run_init_selUnique). The decision phase selects an operator only for a state that has none; every other phase only removes architecture-created elements or adds elements that are not selections.
- **Quiescence is a truth-maintenance fixpoint.** When no new i-supported match and no retraction is pending, every instantiation in the match set still matches the same elements, identified by timetag, and every i-supported match has its instantiation; when no o-supported match is pending, every o-supported match has its instantiation and does not fire again.
- **Retrieval changes no stored knowledge.** A cue-based or non-cue-based retrieval leaves the contents of semantic memory unchanged; only activations and spreading's edge weights may change — the CMC's requirement that retrieval touch only metadata.
- **Storing an LTI changes no other LTI**, and semantic memory's processing never changes architecture-created elements or the goal stack.
- **Episodic retrieval changes nothing stored.** Responding to an episodic-memory command — a cue-based query, retrieve, next or previous — leaves the episode store exactly as it was (not even metadata changes) and changes neither architecture-created elements nor the goal stack.
- **Recording never changes the past.** After an episode is recorded, every earlier episode has exactly the elements it had: the store grows only at the present. The exception is the LTI shown for an identifier value, which Soar 9.6.5 rewrites when the identifier changes its LTI (Section 12); the theorem (newEpisode_past) is stated for episodes without those annotations. Recording changes, among architecture-created elements, only the value of each state's ^present-id.
- **Learning a rule changes only the procedural module's records.** Results with learning on, like those without, leave architecture-created elements and the goal stack unchanged; the identity structure literalizes what it is told to and joining an identity with itself is a no-op.
- **Soar meets the CMC obligations**, including the strict form, because semantic and episodic memory are not yet mapped onto the CMC's declarative module (Section 13).

5. Running Soar programs and testing against Soar 9.6.5

Running a program. soar-lean is a compiled command-line program built from the specification. `soar-lean agent.soar -d 40 --wm` parses the file, runs 40 decision cycles on the formal model, and prints the decision trace in Soar's format, any write output, and the final working memory; `--watch` adds rule firings and retractions, like Soar's watch 3. Agents split across several files are read through source, pushd, popd and cd, as in Soar; agents that use Tcl are first flattened into one file (tools/flatten.py) by loading them into Soar 9.6.5 and printing every production. Small agents run in milliseconds; 40 decisions of the faster Towers of Hanoi variant take under two seconds. The model is written for clarity, not speed, so it is far slower than Soar on large agents. Appendix B is a step-by-step guide to installing, running programs and rule sets, comparing with Soar 9.6.5 and checking the proofs.

Comparison method. Soar 9.6.5 runs in the same workspace as Lean, through its Python package (soar-sml). A script gives the same source to both systems, runs the same number of decision cycles with deterministic indifferent selection (and with chunking off, except in the chunking test sets), and compares the decision traces and the final working memories with identifiers abstracted. Where several operators are mutually indifferent, Soar chooses according to its matcher's internal order, which the manual does not specify; for whole-agent comparisons the script records Soar's choice at each such decision and the model follows it. A chosen operator is identified by its augmentations, an identifier value by its own augmentations one level down, and below that by its ^name or, if it has none, its constant-valued augmentations (two jugs of different volumes are told apart). Everything else — matching, preferences, impasses, support, results, the goal dependency set, learned rules — is checked independently.

Test set	Programs	Result
Hand-written tasks	13	Counting, tie, conflict, constraint failure, state and operator no-change, nested impasses, preference combinations, goal-dependency-set regeneration, an i-supported result kept by a justification, output-link commands, candidate order with a value re-asserted by another rule; all identical
Random preference configurations	300	One to four operators with random acceptable, require, reject, prohibit, best, worst, better, worse and indifferent preferences; all identical
Random multi-cycle programs	100	Eight decision cycles each, with o-supported state variables, i-supported elaborations with negated conditions, operators that change the state, and random preferences; all identical
Soar repository test agents	13	13 identical: arithmetic (40 decisions, 1,024 working-memory elements), blocks world, blocks world with look-ahead, blocks world with operator subgoaling, count test, eight puzzle with look-ahead, Towers of Hanoi (two versions), water jug (two versions), and three basic tests, one using $ notation.
Right-hand-side functions	9	9 identical: every function of the library on chosen arguments, including errors, real numbers to 17 digits, sets reached through one and two attributes, make-constant-symbol and dc (tests/rhslib); and three of the Soar repository's test agents for these functions (BuiltinRHSTests_testString, predict-location, testRHSRand, whose operators proposed in one pass are chosen at random, so it depends on the Rete's order, Section 5)
Random right-hand-side calls	100	Twenty-five random calls each of the mathematical and string functions, on random integers and real numbers, each result also printed with string to 17 digits; all identical
Settings, loading and parser	19	19 identical: source, pushd and popd across directories; max-elaborations (two spellings); max-goal-depth; wait-snc at the top state and below an operator no-change; keep-all-top-oprefs; five programs with rl --set learning-rate, discount-rate, temporal-extension and hrl-discount (with

Test set	Programs	Result
		and without replay); the repository's gp unit test and a gp agent that runs; negations nested three and four deep and inside structured values, compared at five points in the run (tests/settings, tests/rlsettings, tests/parser)
Input from an environment	2	2 identical: a scripted input link whose sensors are added, changed, removed and added again, with real-number and quoted values (tests/input/sensors), and a reactive walker whose environment reads the output link each cycle and moves the agent, removing and remaking the position object (tests/input/walker)

The comparison drove the model. Fourteen behaviors of the first versions were wrong and were corrected from Soar 9.6.5 traces: the order of i-supported and o-supported waves, the effect of a halt, i-supported processing after deselection, i-support for rules that augment the selected operator, o-supported removal of i-supported elements, keeping the selected operator among indifferent winners, the level to which justification conditions are traced, the goal level of unlinked identifiers, condition ordering in the matcher, the per-state memory links, backtracing for results through elements that have since left working memory (found with the semantic-memory tests, but a property of the core), and three found with the episodic-memory tests, also properties of the core: a (state <s> ...) condition tests that <s> is a state, not that it has ^type state (a retrieved episode has ^type state and must not match); an action (<x> ^operator <y>) is an operator proposal only when <x> is a state, and otherwise makes an ordinary element; and a production identical to one already loaded is ignored, as Soar does. Several of these are not stated in the manual (Section 11).

Modeled on the 9.6.5 source. Beyond agreeing with Soar 9.6.5 in behavior, the core follows the structures and control flow of the 9.6.5 source. Preference memory is Soar's: every preference records its instantiation, support, level and goal and is kept in its slot in Soar's order; working memory is made from preference memory in the working-memory phase (acceptable preferences that are not rejected for ordinary slots, acceptable and require preferences for operator slots); o-supported rejects are processed first and remove every preference for the value; results are cloned for the justification or learned rule that supports them while the originals stay with their substate; and semantic and episodic memory add their results through an architectural instantiation, as Soar does. Every element has a timetag, so an element that is removed and re-created is a new element, and an instantiation is identified by the timetags of the elements it matched. The elaboration cycle has Soar's outer and inner loops, with the choice of IE or PE made per state and resumed after an interruption from above, the waterfall rule that holds back a lower state's firing that would change a state already changed in the cycle, and retractions only at the highest active state. Consistency checks are made at minor quiescence and when activity moves to a higher state, an operator is removed in the working-memory phase when its acceptable preference is gone, and the decision starts at the highest state whose operator slot changed and decides the first decidable slot. Soar's condition reorderer is used to place relational tests in explanation records and to build the model of Soar's Rete (below); the matcher keeps its own order, because Soar's is chosen for a Rete network and makes the model's list-based matcher time out on Towers of Hanoi (see the table of ad hoc places in Section 10). On one semantic-memory test (rm3, a rule that removes ^success, which semantic memory restores after every elaboration cycle) Soar 9.6.5 loops to max-elaborations, and the model does the same.

Results and learning when an instantiation is made. In Soar 9.6.5 an instantiation's results are formed, and a rule learned from them, as soon as the instantiation is made (create_instantiation calls learn_rule_from_instance), and the preferences of all the instantiations of a pass of the inner loop are asserted together at its end (assert_new_preferences); working memory changes only in the working-memory phase. The model does the same. When a learned rule is a duplicate, its results are never asserted: as in Soar, the result preferences of the instantiation that made them and the clones of rules learned below it are dropped, and preferences already in preference memory stay; when two instantiations of one pass learn the same rule, only the second one's results are dropped. Because working memory changes only in the working-memory phase, an element keeps its timetag unless it leaves working memory there.

Identifier levels. Soar gives every identifier a goal level: a new identifier takes the level of the instantiation that made it, and at the end of each working-memory phase identifiers are promoted to the highest state that links them and demoted when those links go (do_promotion, do_demotion), where links are the values of elements and of preferences; identifiers made by deep-copy are never promoted. Which elements are results, which conditions of a learned rule are grounds and which are local, and which identifiers a rule repair must connect all depend on these levels, and the model keeps them as Soar does. Two further details follow the 9.6.5 source: the walk that collects the structure going with a result follows preferences as well as elements, without a bound (add_results_if_needed); and an identifier that a rule made new does not need repair (reorder_action_list counts only actions on identifiers that were not unbound right-hand-side variables).

The Rete's order. Which matches exist does not depend on Soar's matcher, but the order in which a pass fires them does: Soar puts each new match at the head of its state's assertion list when the Rete reaches the rule's production node (p_node_left_addition), and the preference phase takes matches from the head (postpone_assertion), newest first; retractions likewise. The order a pass makes its instantiations in decides the order of the operator candidates, and so the choices of every selection policy, which of two instantiations making the same value supplies the element's preference (and so what chunking traces back to and which elements receive working-memory activation), and the order of the elements made next. The model includes a model of Soar 9.6.5's Rete network for this order (Soar/Rete.lean, following rete.cpp function by function): each rule's conditions in Soar's order, with its goal tests where Soar's reorderer puts them (on the first condition for the state); nodes shared between rules, beta memories merged with their single join and split when a second join is added; negative and conjunctive-negation nodes; every memory newest first; left and right unlinking and relinking (before the nearest linked ancestor on the same alpha memory, or at the end); new alpha memories filled from the elements already present; and tests on variable locations, so that rules with different variable names share nodes. Elements enter it in the order Soar makes them, and the model now numbers elements in that order, so its timetags are Soar's: in a working-memory phase, slots in the reverse of the order they were first marked changed, values in candidate order, operator acceptable-preference elements after the others (decide_non_context_slots, do_buffered_acceptable_preference_wme_changes); a new substate's elements in create_new_context's order; impasse items in candidate order, their counts made again at every update; each slot's preferences in the order Soar adds them (assert_new_preferences: the newest instantiation first, each one's preferences last made first), which decides the order of a slot's values and which preference supports an element. Removals leave the Rete in the reverse of the order Soar queues them: elements of slots decided, then a removed operator's and substate's elements, then garbage collection. Since October 31 the model of the Rete is also the matcher: each match it adds is rebuilt from the timetags of its elements (St.reteMatchOf), and the match set is kept from one elaboration cycle to the next and changed only where the network added or

removed matches (St.reteCache). The model's own matcher (St.matches), the one the soundness and completeness theorems are about, is still used for rules with long-term-identifier tests, which the network does not evaluate, and with --no-rete; soar-lean --check-match compares the two on every cycle (the matches, their number, bindings and support), and they agree on the tests. On the test-suite agents, tools/firecmp.py finds the same rule firings in the same order as Soar 9.6.5, and tools/ttcmp.py the same timetags and identifiers for every element, except on one agent whose rules copy architecture elements (the model keeps one element for two of the same content); with the Rete's order, all 13 agents, all reinforcement-learning programs and testRHSRand match without Soar's choices replayed. The network's size is limited to 400,000 tokens: a rule with many similar conditions can make partial matches without bound (the Towers of Hanoi test agent, which makes Soar 9.6.5 run out of memory a decision after the decisions tested), and beyond the limit the model's own matcher and the earlier ordering are used for the rest of the run.

Support as the Rete decides it. Soar's Rete decides whether a new match is i- or o-supported when the match is added (p_node_left_addition), and it does so by looking at every current match of the production, not only the new one: an action on an identifier other than the selected operator of the deepest state a match tests makes the new match o-supported, with the action's identifier always taken from the new match. An operator elaboration is therefore o-supported whenever another match of the same rule has a different selected operator, as happens with nested selection states (Section 12). The model applies the same classification once, when its Rete adds the match, against the production's matches the network holds at that moment, newest first (St.reteCache, Net.sup).

The right-hand-side function library. Every function Soar 9.6.5 provides is modeled from its source (rhs_functions.cpp, rhs_functions_math.cpp; Soar/Rhs.lean), except from-st-xml, which reads outside input; accept, which reads standard input, behaves as Soar does at end of input and gives no value; @ and link-stm-to-ltm are described with semantic memory (Section 7). Functions of their arguments follow the C code step by step: C's truncating conversions and integer division, the 32-bit int in extrapolate-x-position and extrapolate-y-position, Soar's value of pi (3.141592653589) in the heading and distance functions, the unsigned 64-bit binomials of compute-dice-probability, and strtoll and strtod when int and float are given strings. Real numbers are printed as Soar prints them, with six digits after the point and with 17 for string, computed exactly from the binary value. Functions that read the agent see working memory in Soar's order (an identifier's slots and their elements, most recent first): size, sum, product, the set functions and select-point-closest-to-vector. dc gives the decision count; make-constant-symbol keeps Soar's counter and avoids every constant of the source, of working memory and those it has made.

Two behaviors of Soar's symbol table show in the results. A real-number constant is found by ==, so 0.0 and -0.0 are one symbol, the one made first: a function whose result is a zero gives the zero already present, with its sign. And a NaN is the processor's default NaN, printed -nan. The model does both (RState.canon). Three behaviors are kernel defects (Section 12): sum and product read the integer field of a real-number value, so 0.1 counts as -1717986918; set functions return their error as a constant whose name includes the bars (|ERROR: First parameter needs to be an identifier.|); and select-point-closest-to-vector reads only the first slot of its identifier.

Settings, loading and the parser. soar-lean follows source, pushd, popd and cd as Soar 9.6.5's command line does: a file is read from the current directory, and while it is read the current directory is the file's own. It reads the soar settings max-elaborations, max-goal-depth (a no-change substate deeper than it halts the agent at once, before the application phase, as create_new_context does), wait-snc (a state no-change makes no substate; Soar's trace then prints the waiting state's own impasse at every

decision, which the model reproduces) and keep-all-top-oprefs, and rl --set. o-support-mode is not a setting of Soar 9.6.5. gp is expanded as Soar expands it: each bracket list is replaced by each of its values, the first list varying fastest, and the rules are named name*1, name*2, ...; values are separated by white space, a |...| constant is a value of its own even when not separated, and an escaped space is part of a value. Rules that differ only in a numeric-indifferent value are duplicates (Section 12), so gp lists of such values load one rule. On attributes other than ^operator Soar 9.6.5 keeps only acceptable and reject preferences, ignoring others with a warning, and rejects a rule with a binary one; the model does the same. Negations may be nested to any depth (Cond.nccg, a negated conjunction given as its nodes in preorder), also inside a structured value of a negated condition.

Input from an environment. Every decision cycle begins with the input phase, in which the environment gives the elements now on the input link (an Env's input function, of the decision count and the output link's current elements). The elements it gives replace those it gave before: one that is given again keeps its timetag, one that is no longer given is removed, and identifiers the environment makes come from a range the agent does not use (soar-lean numbers a script's identifiers from 1,000,000 up). soar-lean reads the changes from a script (--input F), lines K + ID ^attr VALUE and K - ID ^attr VALUE made before the input phase of decision cycle K, where ID is the input link (I2) or an identifier the script names (#name). For a reactive environment, tools/simenv.py runs the environment's step function, a Python function of the cycle and the output link, against Soar 9.6.5 through SML, records the changes it makes and gives the same script to the model, so the model is tested on the input Soar received. Soar adds input-link elements through SML's input functions, called here before each input phase, which is where the model makes the same changes.

Support of memory-system results. Semantic- and episodic-memory results are preferences of an architectural instantiation, never in the match set and so never retracted; their support is the kind of the last elaboration cycle (add_pref_to_inst sets o_supported from FIRING_TYPE): o-support if it was a PE cycle, i-support if it was an IE cycle. The model tracks the kind (St.firing) and does the same. tools/memsupport.py compares the support of every element on a result link with Soar's preferences command: 1201 of 1202 memory programs agree, both kinds occurring (2496 i-supported and 257 o-supported elements); the one remaining case is an agent that uses ^result for its own structures, which the script takes for a result link.

6. Reinforcement learning and exploration

Reinforcement learning (manual Chapter 5) is formalized as an extension of the decision cycle. A rule is an RL rule when its only action is a numeric-indifferent preference with a constant value. When an operator is selected, the RL rules whose preferences contributed to its Q-value are recorded for that state. When the next RL operator is selected in the same state, those rules are updated by Sarsa, with the change divided among them in proportion to their contribution, and with each value kept as two parts, the expected current reward and the expected future reward, as Soar does. Reward is read from each state's reward link. Gaps (selections of non-RL operators between RL operators), operator no-change impasses, hierarchical discounting, state retraction and halting are covered.

Element	Formalization
Learned values	Kept in procedural module state, per rule; every live instantiation of an updated rule takes the new value at once, as in Soar. soar-lean --rl --rl-values F writes them, with each rule's update count, to F in the same form as Soar's print --rl.

Element	Formalization
Reward collection	For every state, whenever an operator is selected in any state; a decision that creates an impasse collects nothing (Section 11). Rewards are discounted by the gap age and, if hrl-discount is on, by the hierarchy age.
Update on state retraction	Reward is collected for the retracted state and only the expected current reward is updated, with no next state. Substates are removed as soon as the operator above them is deselected, as in Soar 9.6.5, which fixes when this update happens.
Update on halt	Reward is collected and both parts are updated with a next value of 0 (Section 11).
Learning policies	sarsa (the default); q-learning, whose target is the value of the best candidate rather than the one selected; on-policy-gq-lambda and off-policy-gq-lambda, which keep a secondary weight per rule (step-size-parameter η) and correct the expected future reward of the rules behind the operators now proposed, the off-policy form weighting the traces by the ratio of the target policy's probability to the exploration policy's (ExploreCfg.rho)
Eligibility traces	eligibility-trace-decay-rate λ and eligibility-trace-tolerance: with λ > 0 the traces of earlier selections decay by λ times the discount and are dropped below the tolerance, and every rule with a trace is updated; a state with a substate adds no trace
Decay modes	normal (fixed learning rate), exp (1 / (n + 1) after n updates), log (1 / (ln(n + 1) + 1)) and delta-bar-delta, which adapts a learning rate per rule (its β and h, Soar's initial values −3 and 0, with meta-learning-rate θ)
Template rules	A :template rule's matches make RL rules (rl*<template>*N, numbered as Soar numbers them, skipping a duplicate), whose conditions are the template's instantiated with the match, identifiers as variables, named as Soar's variablization names them, and other values as constants, and whose value is the template's. As in Soar, a new rule is added to the Rete at once and its matches fire next, in the same elaboration cycle (rl_build_template_instantiation, postpone_assertion), also in substates while chunking; and as in Soar, an identifier keeps the variable an earlier template rule gave it, so two identifiers can share one variable (kernel defect 28)
Parameters	learning, learning-rate (0.3), discount-rate (0.9), temporal-extension (on), temporal-discount (on), hrl-discount (off), learning-policy, eligibility-trace-decay-rate (0) and -tolerance (0.001), decay-mode, step-size-parameter (1), meta-learning-rate (0.1), with Soar 9.6.5's defaults
Not formalized	RL updates of rules learned by chunking

Proved. RL changes only values: an update leaves architecture-created elements, the instantiations and the slots unchanged, and changes preference memory only in the numeric values of preferences, so no rule's conditions, bindings, created elements or preference structure change. The existing theorems still hold with learning on, among them that rules never change architecture structures and that a decision adds at most one operator selection.

Tested. Comparisons run with learning on and epsilon-greedy exploration (epsilon 0.3, fixed seed); in the regression Soar's exploratory choices are recorded and replayed, while the Q-values that drive them, the updates and the final values are computed independently (without replay, see Exploration below). The test set is the repository's water-jug RL agent, four hand-written tasks (a bandit, a gap, a halt and a hierarchical task), a second hierarchical task with RL operators at both levels and rewards on both reward links, and 60 randomly generated programs with several RL rules per operator, shared rules, negative rewards, and gaps from non-RL operators. All 66 match Soar 9.6.5 in decisions, final working memory, the number of updates of each rule, and each learned value. The variants are tested on 60 programs (tests/rlvariants): six tasks (a bandit, a gap, a two-level hierarchy and three of the random programs), each run with q-learning, λ = 0.5 and 0.9, q-learning with λ = 0.5, on- and off-policy GQ(λ), the exp, log and delta-bar-delta decay modes, and temporal-discount off; and template rules on 14 programs (tests/rltemplates): a walk learned with one template, also with λ = 0.6, templates whose value comes from a variable, and templates in substates with chunking on, among them rules whose identifiers share a variable (kernel defect 28). All match Soar 9.6.5 in decisions, final working memory, update counts and learned values (63 of the 74 also without replaying Soar's choices, with the order in which a pass fires its matches taken from the model of the Rete, Section 5).

Two behaviors of Soar's update, both reproduced, matter with the variants (Section 12). With eligibility traces, every rule with a trace is updated, but the value Soar then gives the preferences of all their current instantiations is new_combined, a variable left over from the update loop: the new value of the last rule updated, not each rule's own. And with GQ(λ), the value print --rl shows, and that new instantiations take, is the rule's right-hand-side value, set when the rule is updated; GQ(λ)'s later correction of the expected future reward changes the rule's stored value but not that right-hand side (St.rlRef).

Exploration and random numbers

When preference resolution ends with several mutually indifferent candidates, the policy set with decide indifferent-selection picks one (Soar 9.6.5 exploration.cpp; Soar/Explore.lean). first and last take the first or last candidate; softmax (the default) picks at random in proportion to the positive values, or uniformly if none is positive; epsilon-greedy picks at random with probability epsilon and otherwise one of the highest-valued candidates, at random among ties; boltzmann picks with probability proportional to exp(value / temperature). A candidate's value is the sum of its numeric-indifferent preferences and the numbers referred to by its binary-indifferent ones (0 for a referent that is not a number), or with numeric-indifferent-mode --avg their average; a candidate with neither has value 0. With auto-reduce on, epsilon and the temperature are reduced after each decision phase, exponentially or linearly, a reduction being kept only if the result is a valid value. The same value is the Q-value reinforcement learning uses.

The random numbers come from Soar's generator, Wagner's MT19937 (soar_rand.h), reproduced bit for bit (Soar/Rand.lean): seed is MTRand::seed, an integer in [0, n] is drawn by masking and rejection, and a real number in [0, 1] is the next word divided by 2^32 − 1. srand sets the seed. The right-hand-side functions rand-int and rand-float draw from the same generator, evaluated in the order of the actions and, within an action, identifier, attribute, value, referent.

Proved. resolve_none: if preference resolution does not reach the choice among mutually indifferent candidates, every choice function gives the same decision. resolve_some: if it does, with candidates d, the winner is what the choice function returns on d. Together they say the policy affects only the choice among indifferent candidates, and that its pick is the winner.

Candidate order. The random pick, epsilon-greedy's ties and first and last depend on the order of the candidates. Soar builds the list by walking the operator slot's acceptable preferences, most recently asserted first, putting each operator onto the front of the list, and it asserts the preferences of one pass of the inner elaboration loop in the reverse of the order their instantiations were made (assert_new_preferences walks newly_created_instantiations, newest first). So operators proposed in earlier passes come first, oldest first, and those proposed in one pass come in the reverse of the order they fired. The model does the same (St.candOrder), with the order in which a pass fires its matches taken from its model of Soar's Rete (Section 5).

Tested without replay. 17 programs written for the purpose (tests/explore) exercise every policy, both modes, all three reductions and the random right-hand-side functions, with operators proposed in successive passes so that the candidate order is fixed; all match Soar 9.6.5 in decisions and final working memory, including every random number drawn. The reinforcement-learning programs, run with epsilon-greedy exploration and seed 42, also match without replaying Soar's choices: all 66. With --first and no replay, all 13 test-suite agents match (9 before the candidate order was modeled, 11 before the Rete's order was). Before the Rete's order was modeled, the water-jug RL agent, the two template walks, arithmetic and blocks world with look-ahead differed without replay, because operators proposed in one pass fired in a different order.

CMC mapping. In the CMC, reinforcement learning tunes procedural memory as a learning step separate from the cycle, computed from procedural memory and a snapshot of working memory. Soar's update needs more: which rules contributed to the previous selection, the reward accumulated since, and the gap and hierarchy ages, all of which are procedural module state, and it happens inside the decision. The model therefore keeps learned values in procedural module state, and the CMC learning step is the identity. This was decided on October 29, 2026; chunking's learned rules are kept the same way, and whether CMC.Proc should let the cycle itself change procedural memory is left to its next revision (Section 13).

7. Semantic memory

Semantic memory (manual Chapter 6) is formalized in two parts. Soar.SMem is the long-term store: long-term identifiers (LTIs) with augmentations, storing, cue-based queries and reading an LTI. The store follows Soar 9.6.5's SQLite schema, because the order in which a query considers candidates — which decides between equally activated matches — follows it: each augmentation is a row with a row number and an activation, and a query walks the rows of its most selective cue element by decreasing activation and, among equals, decreasing row number. The second part, in the decision cycle, reads commands from each state's ^smem.command, places results under ^smem.result, and keeps the links from short-term identifiers to LTIs.

Element	Formalization
When commands are processed	Storage after every elaboration cycle of the proposal and application phases; retrievals at the end of the output phase; states from the bottom of the goal stack up. A command is processed when what the pass looks at has changed; a change first removes the state's status elements (Section 11).
Storage	^store replaces an LTI's augmentations (creating the LTI if needed) and links identifier values to LTIs without storing their contents; ^store-new and ^link-to-new-ltm as Soar 9.6.5 implements them (Section 11).

Element	Formalization
Retrieval	Cue elements with constant, LTI-linked and unlinked-identifier values; ^prohibit, ^neg-query, ^math-query (less, greater, less-or-equal, greater-or-equal, max, min), ^depth, ^link-to-ltm; ^retrieve. Retrieved LTIs are placed on new identifiers; status elements ^success, ^failure, ^bad-cmd, ^retrieved, ^depth-retrieved.
Activation	Recency (the default), frequency and base-level, as Soar 9.6.5's lti_activate and lti_calc_base compute them; activate-on-query and activate-on-add as in 9.6.5 (stores do not count as accesses, and smem --add does not activate).
Base-level activation	Each LTI keeps its number of accesses, its first and last access and a history of its last ten accesses; its activation is log(s / (1 + s)) for s = Σ (t − t_i)^(−d) over the history, t counting accesses (base-decay d, default 0.5); with no history, one access at the agent's creation is assumed, base-unused-age-offset earlier. Policies: stable (recomputed when accessed), naive (the query's candidates are recomputed before its search, in order of LTI number) and incremental (at each access, the LTIs last accessed exactly one of base-incremental-threshes ago are recomputed). base-inhibition adds log(q / (1 + q)) for q = (1 + (r/10)^−1)^−1 and r the age of the latest access. A ^prohibit marks the LTI, and its next activation takes back the latest access.
LTIs with thresh or more augmentations	Their augmentations are at the highest activation (SMEM_ACT_MAX), and a query takes them in the order of the LTI's own activation, from a priority queue merged with the other candidates. The queue and the order in which cue elements are taken (by weight) reproduce libstdc++'s heap, so ties come out as in Soar.
smem --remove and smem --query	smem --remove takes values or whole attributes from an LTI (or all its augmentations) and writes it again without activation; smem --query has no effect but the naive policy's recomputation (the model does not print what it finds).
Command validity	Soar decides it by reading the command's elements in its internal slot order, so the model tracks Soar's slot order (Section 11).
Results and justifications	Status and retrieved elements depend on the command elements, as Soar's architectural instantiations do, so results built from retrievals in substates get the right justifications.
smem --add	Clauses, dot paths, multiple values, @n and aliases, variables local to each command.
Long-term-identifier tests	In conditions: { <x> @+ } (the value is linked to an LTI), @- (it is not), @ <y> or @ n (linked to the same LTI as <y>, or to LTI n) and !@ (not); a constant is never linked. The links are bindings the matcher starts from (@n ↦ the LTI of identifier n), so the soundness and stability theorems keep their form. In actions: (@ <x>), the number of the LTI <x> is linked to (no value if none), and (link-stm-to-ltm <x> n), which links <x> to LTI n when the instantiation is made, if n exists (set_lti_id_rhs_function_code).
Spreading activation	Sources are the LTIs with elements in working memory, counted under the LTI an identifier stood for when each element arrived. Each source's fingerprint is Soar's best-first walk along edge weights (1 / the number of

Element	Formalization
	LTI-valued augmentations), each step times the continue probability, cut at the baseline, the depth limit and spreading-limit recorded paths (loop avoidance by (LTI, weight) pairs, as Soar compares them), with children in SQLite's order and the queue as libstdc++'s heap; its spread to each LTI is normalized by the sum, added left to right as SQLite 3.40 sums. When a query's first cue element has 2 or more rows, pending sources enter or leave the context and each candidate's spread is recomputed: per source, the larger of the floor b / (p(1+p)^D) (b the baseline, p the continue probability, D the depth limit) and the source's factor times its share, the factor being q / (1 + q) for q the age term (t + base-unused-age-offset)^(−d) or, with spreading-wma-source, the sum of the time sums of the source's elements' decay histories (falling back to the age term when that sum is 0); the candidate's activation is its base plus log(spread) − log(floor). Queries take LTIs receiving spread from a queue on that activation. Edge updating: references from working-memory activation move edge weights toward 1 per cycle and renormalize, and invalidate the paths through the parent, through Soar's invalidation check, which is never reset.
Database on disk	smem --set database file, path and append: the store is written to the file when the agent ends, and a later agent opening it finds what Soar keeps in its database (SMemDB.reopen: the LTIs and augmentations with their row order, activations, histories, prohibitions, symbol table, aliases, clock and the spreading paths; the stored thresh and activation-mode replace the new settings; init_db empties the current spread). With append off the store starts empty except for the parents marked for invalidation, which Soar does not drop (SMemDB.reopenEmpty, kernel defect 29). smem --backup F writes a copy of the open store; smem --clear empties it; smem --init, before a run, changes nothing the model holds. soar-lean writes its own text format (Soar/Store.lean), not SQLite, so its files and Soar's cannot be exchanged.

Proved. Retrieval changes no stored knowledge, only activation, and computing spread changes no stored knowledge, only activations and edge weights; storing an LTI changes no other LTI; and semantic memory's processing never changes architecture-created elements or the goal stack (37 theorems with their lemmas).

Tested. 759 of 760 programs match Soar 9.6.5 in decisions, final working memory, and the contents and activations of semantic memory (compared up to renaming of LTIs, Section 12): 17 hand-written tasks, among them the command-order cases; 300 random programs with stores, store-new, queries with every modifier, retrievals and malformed commands; 200 random programs that add substates with their own commands and results, commands made by i-supported rules, and rules that change retrieved structures; one program that uses every long-term-identifier test, the @ function and link-stm-to-ltm (tests/lti); 200 random programs with base-level activation (tools/gen_smembl.py, tests/smembl), under every update policy with random decay, offsets, thresholds, inhibition and small values of thresh; 4 programs for smem --remove, smem --query and LTIs with more than 100 augmentations (tests/smemcli); and 37 of the 38 semantic-memory functional tests in the Soar repository, among them the three base-level tests. Spreading activation: 150 of 150 random programs (tools/gen_spread.py, tests/smemspread) match, with random continue probabilities, baselines, depth and spreading limits, loop avoidance, update policies and, in 66 of them, working-memory activation, 53 of these with

spreading-wma-source; the store, including every spread activation, matches in all 150. (Eight of them differed until October 31 by one reference on elements made from a malformed command's status: the model counted a query element among the elements the bad-command result depends on, which Soar does not; doc/DIFFERENCES.md.) The functional test AlphabetAgentAllOn, with edge updating and spreading-wma-source, matches in decisions, the store and all spread activations, and differs in four working-memory activations, where soar-lean reports reads Soar makes of freed memory. With edge updating, 21 of 30 random programs match; Soar's edge updating reads an identifier field of constants (Section 12), so its results depend on memory contents.

Tested on disk. 68 programs run in two agents (tests/dbfile, Section 5): the first leaves its semantic or episodic store in a file, the second opens it and continues; all match Soar 9.6.5 in decisions, working memory and both stores. They cover stores made by smem --add and by store commands, queries and retrievals of the first agent's memories, activation with the stored threshold and activation mode, append off, smem --backup and --clear, spreading activation continued from stored paths, episodes continued and queried across agents, identifier edges re-used from the stored pools, and reruns of 56 programs of the other memory sets.

CMC mapping (deferred). The semantic store is held for now in the procedural module state, and its command and result structures are placed in the region, so the CMC obligations hold but say nothing about semantic memory. Section 13 records the preferred mapping onto the CMC's declarative module, deferred on October 29.

8. Episodic memory

Episodic memory (manual Chapter 7) is formalized in two parts. Soar.EpMem is the episode store, the encoder that records episodes, and the retrieval algorithms; the decision cycle reads commands from each state's ^epmem.command and places results under ^epmem.result. The store and the encoder follow Soar 9.6.5's schema and source, not only the manual, because a cue-based query's result depends on more than the content of each episode: Soar walks backwards through time over the intervals during which the stored structures that could match the cue were present, and scores an episode only where one of those intervals begins or ends. Which structures count as the same structure over time — an identifier that is replaced by a new one in the same place keeps its node, for example — therefore decides which of two equally good episodes is returned and whether a graph match is tried, so the model reproduces Soar's identification of nodes and edges.

Element	Formalization
Recording	At the end of the output phase (or after the decision phase with phase selection); every decision cycle with the dc trigger (the 9.6.5 default, Section 11) or when the output link has a new element with the output trigger; force remember and ignore; exclusions (by default ^epmem and ^smem). The top state's working memory is recorded breadth-first; acceptable operator preferences are recorded as ^operator*. After recording, every state's ^present-id is the next episode number.
The store	Nodes for identifiers and edges for elements, each edge with the intervals of episodes in which it was present. The encoder is Soar's: identifiers keep their node while they stay in working memory, a new identifier re-uses the node of one that left the same parent and attribute, and edges are opened and closed incrementally. It is applied once per output phase to the net

Element	Formalization
	change in working memory.
Cue-based retrieval	A port of Soar's interval search: the cue's literals (with shared and nested identifiers and cycles), partial and unique edges, scoring by satisfied leaf literals of the ^query and ^neg-query cues, ^before, ^after, ^prohibit, and the graph match with its mapping; graph-match-ordering undefined, dfs and mcv.
Other retrievals	^retrieve by episode number, ^next and ^previous relative to the state's last retrieved episode; ^retrieved no-memory, ^success, ^failure, ^status bad-cmd.
Results	The retrieved episode on a new identifier under ^retrieved, with its meta-data (memory-id, present-id, match-score, match-cardinality, cue-size, normalized-match-score, graph-match, mapping); like semantic memory's, they depend on the command and cue elements. A command is processed again when anything reachable from it changes, after the old result is removed.
LTIs in episodes	An identifier value that stands for an LTI is recorded with it per interval, as Soar does: an interval still open carries the LTI the value had when the interval opened, a closed one the LTI it had when the element left. An identifier that changes its LTI (by link-stm-to-ltm, say) therefore shows the new LTI in earlier episodes too, once the interval closes, and one node can show as an LTI under one parent and not under another in the same episode, as epmem --print shows it.
merge and lazy-commit	Accepted. Neither has an observable effect in Soar 9.6.5: merge add sets a flag that is already always true for every reconstructed identifier, and lazy-commit only groups database writes into one transaction.
Balance	Below 1, cue literals are weighted by their elements' working-memory activation (Section 9).
Database on disk	epmem --set database file, path and append (default off): the episodes are written to the file when the agent ends; a later agent with append on continues from them (EpDB.reopen, following epmem_init_db: every open interval is closed at the last episode with the LTI it opened with, the identifier edges' pools are rebuilt from all stored edges, the episode count continues, and only the top state is known). epmem --backup and --init at load time are not modeled: Soar's episodic database is not open then, so --backup writes nothing, and --init opens it as the first episode would.

Proved. Responding to any command leaves the episode store unchanged, together with architecture-created elements and the goal stack; recording an episode never changes an earlier episode, apart from the LTIs shown for identifier values; and recording changes only the ^present-id values among architecture-created elements (43 theorems with their lemmas).

Tested. 671 programs match Soar 9.6.5 in decisions, final working memory, and every stored episode (compared with epmem --print, up to renaming of nodes): 3 hand-written probes; 300 random programs in which objects change, are replaced, gain and lose parts and become shared while queries with every modifier, retrievals and next and previous commands run; 300 random programs that add several objects under one attribute (so graph matching matters), cues with shared identifiers and operators,

commands from substates, and random settings of graph matching, its ordering, the trigger and exclusions (all 300 generated; 11 of them contain a rule loop that runs to max-elaborations at the top level while a substate exists, which match because the model has Soar's inner elaboration loop; two, f229 and f299, match only some runs of Soar 9.6.5, whose results for them differ from run to run because the graph match tries mappings in memory-address order, kernel defect 4 — Section 12); 24 of the random programs run again with merge add or lazy-commit off (tests/epsettings); and 44 of the 44 episodic-memory functional tests in the Soar repository, among them the fourteen 110-episode unit tests and the Hamiltonian-path graph-match tests. Those with working-memory activation are also counted in Section 9; two more tests need the spatial-visual system and were not run; the two that keep an episodic database on disk (the Hamilton store and retrieve pair) run with their file settings restored in tests/dbfile. Also, and not counted above, 3 programs in which an identifier changes its LTI while recorded, through two parents and after leaving working memory (tests/epltichange), match.

CMC mapping (deferred). For now episodic memory is held like semantic memory: the store in procedural module state, its structures in the region. The preferred mapping for both, deferred on October 29, is set out in Section 13.

9. Working-memory activation

Working-memory activation (manual §9.3.2.1, the wma command) gives each o-supported working-memory element a decay element: a history of the decision cycles in which the element was referenced and how often. An element's activation in cycle t is log $\Sigma\, n_i\, (t - t_i)^{(-d)}$ over the history (at most ten entries; d is the decay rate, 0.5 by default, which Soar stores negated), and an element whose activation falls below the decay threshold can be forgotten. Soar.WMA holds the data and the arithmetic; the decision cycle (Soar.Cycle) says when elements are referenced and when they are forgotten. The model follows working_memory_activation.cpp and its call sites in decide.cpp, preference.cpp and run_soar.cpp, because the manual says little about which events count as references, and the activations Soar prints depend on every one of them.

Element	Formalization
Decay elements	Only o-supported elements have one; architecture-created elements, input elements and i-supported elements never do. The history keeps ten entries of (cycle, references), with the total number of references and the cycle of the first; Petrov's approximation (petrov-approx) adds the references beyond the ten. An element with no decay element has activation 1. print shows the activation with C's %0.2g, and soar-lean writes it the same way in its working-memory file.
References	At the start of every preference phase, each element tested by each pending match gets a reference: the matches about to fire and those still waiting, such as o-supported matches during the proposal phase. Each condition counts, so an element tested by two conditions gets two references. The state keyword tests the identifier and is not a condition. Each acceptable preference a rule asserts references the elements of its slot that have its value. A preference for a value with no element yet counts a reference for that value; the count is given to the element when it is made, and it is cleared when the slot is first changed in an elaboration cycle. An i-supported element passes its references to the o-supported elements its instantiation depends on (its o-set, made once, through other i-supported elements). A selected operator does the same through its

Element	Formalization
	winning preference, and a memory system's results do it through their command and cue.
Initial boost	A new decay element starts with the floor of the mean time sum of the elements tested by the instantiations of the element's o-supported acceptable preferences. An element whose decay element was just made is left out; an i-supported element stands for its o-set; an element with no preference counts e.
Clock and histories	The activation clock starts at 1 and advances at the end of each output phase. In the output phase, histories are updated after semantic-memory retrievals and again after episodic memory, and episodic memory runs with the clock one ahead, so that its cues see the activations of the next cycle. Forgetting comes last.
Forgetting	naive: every element whose activation is below the threshold and that was not referenced in this cycle. bsearch and on: each element is queued at the cycle in which its activation will fall below the threshold, found by doubling and binary search in Soar's unsigned 64-bit cycles (on first estimates it from each history entry's references), and requeued if it is still above the threshold when that cycle comes. A forgotten element loses its o-supported preferences, and a working-memory phase follows. forget-wme lti forgets only elements of identifiers linked to an LTI, and with bsearch and on only all of that identifier's o-supported elements together; fake-forgetting forgets nothing.
Episodic memory	With balance below 1, each literal of a cue is weighted by its cue element's activation instead of 1, which changes match-score and normalized-match-score and can change which episode is retrieved.
Settings	wma --set activation, decay-rate, decay-thresh, petrov-approx, forgetting (off, naive, bsearch, on), forget-wme (all, lti) and fake-forgetting. As in Soar, the others cannot be changed while activation is on, and decay-rate and decay-thresh are stored negated. timers and max-pow-cache are accepted; they do not change results.
Elements out of working memory	An instantiation keeps the elements it tested, and their decay elements, after the elements leave working memory, and these still count in initial boosts. Where Soar reads the stale preference of such an element (Section 12), the element contributes nothing, and soar-lean reports how many such reads occurred, so that a difference from Soar can be traced to this cause.

Proved. The activation state is not an architecture structure: setting it changes no architecture-created element and no goal. The theorems that an elaboration cycle and semantic memory's processing never add architecture structures hold with the new steps (two lemmas).

Tested. 324 of 325 programs match Soar 9.6.5 in decisions and final working memory, with every element's printed activation compared: 1 hand-written program; 200 random programs (tools/gen_wma.py, tests/wmafuzz) in which operators add, change and remove attributes of several objects while elaborations test them and the counter, under random decay rates, thresholds, Petrov's approximation and every forgetting mode, 32 of them forgetting elements; 60 semantic-memory programs (tests/smemwma) and 59 of 60 episodic-memory programs (tests/epwma) with activation on,

most of the latter with balance below 1; and the 4 working-memory-activation tests in the Soar repository: the activation functional test, the two episodic encoding tests with activation, and testWMActivation_Balance0. The remaining episodic program differs in a graph-match mapping that Soar takes in memory-address order (Section 12); with graph-match-ordering dfs it matches. Thirteen small probe agents were also compared decision by decision with Soar's wma --history.

Limits of the comparison. The random generator avoids by default a rule that makes the same o-supported value twice in one cycle, or the same i-supported element from two instantiations: which preference Soar keeps, or which one supports the element, follows the order of the Rete and of preference assertion, and with activation on it decides which elements receive the references. With both orders modeled (Section 5), 100 programs that make such preferences (tools/gen_wma.py with DUPS=1, tests/wmadup) are compared as well; 100 match, and with the earlier ordering 29 of a first 40 matched. The generator still avoids an i-supported element kept across a change of an o-supported element that its instantiation tested, where Soar reads a freed preference (Section 12).

10. Chunking

Chunking (manual Chapter 4) is formalized in two parts. Soar.Chunk is the learning computation: given the records of the instantiations that fired in substates and the results of one instantiation, it computes the rule Soar 9.6.5 learns. The decision cycle keeps a record of every instantiation that fires in a substate while learning is on (its rule, bindings including the new identifiers it made, the element each condition matched, its negated conditions and what it made), calls the computation when results are created, adds the learned rule to production memory, and gives the results to the learned rule's instantiation. The model follows the explanation-based chunking of the 9.6.5 source rather than only the manual, because which elements become variables depends on details the manual leaves open.

Element	Formalization
Identities	Each variable of each instantiation, and each new identifier it makes, has its own identity; constants have none. Conditions on a substate's ^superstate element share one identity for the state and one for the superstate, per substate (Soar's floating identities).
Dependency analysis	From the instantiation that made the result: conditions on elements at or above the result's goal level (the match goal's level minus one) are grounds; other conditions are traced through the instantiation whose preference supported the element when the condition's instantiation fired (Soar's bt.trace, recorded at that moment: after each elaboration cycle Soar points an element at the first acceptable preference in its slot's list), and the condition is unified with that instantiation's action: two identities are joined, an identity facing a constant is literalized, and the arguments of the right-hand-side functions Soar registers as literalizing (the mathematical functions, compute-range among them, and string) are literalized. The selected operator traces to the preference that won the decision (a require if there was one), a substate's ^item to the superstate's acceptable preference. Local conditions on a state with no creator are dropped; a ^quiescence t test, also carried through justifications, makes a justification instead of a chunk.
Singletons	Two grounds that matched the same element have their value identities joined. With automatically-create-singletons on (the 9.6.5 default) every

Element	Formalization
	element with a symbolic attribute counts; otherwise only the architecture's (type, superstate, impasse, attribute, smem, epmem, reward-link).
Constraints	Relational tests and disjunctions of local conditions are moved to the first ground that tests the same identity, inverted if they must be attached to the other side; tests of the grounds are kept. Which condition of a rule carries a relational test is where Soar's reorderer puts it (reorder_lhs: tests taken out, conditions ordered by Soar's cost estimate with one-step lookahead and canonical tie-breaking, tests put back on the first condition at which both variables are bound).
Operator selection knowledge	With add-osk on (the 9.6.5 default), the preferences that decided a state's operator (prohibits and rejects, the better and worse preferences of the surviving candidates, best and worst preferences that mattered, the winner's numeric and binary indifferences) are kept when it is selected; an instantiation that tests that operator, and the one that proposed it, are backtraced together with the instantiations that made them, without identity joins. Prohibits of any operator tested are always backtraced.
Rule repair	A rule with an identifier that is no condition's value and not a state, or an action on an identifier nothing binds, is repaired as in 9.6.5: elements on a path from the state at the identifier's level (Soar's depth-first walk over rule-made elements) and missing ^superstate links become conditions; if that fails, a justification is made.
Memory retrievals and deep-copy	A semantic- or episodic-memory retrieval leaves an explanation record whose conditions are the state's link, command and result elements and the command's cue, as Soar's architectural instantiation for a memory system has them; a rule learned through it tests the cue and makes the retrieved structure. deep-copy copies the structure under its argument; the instantiation also tests each element copied and each copy has its source's identity, so a learned rule recreates the structure on the source's variables.
Rule formation	Each joined identity becomes a variable; a literalized one the constant matched (other tests of that element are then dropped). Conditions with the same three elements are merged and their disjunctions intersected. Negated conditions whose identifier is tied to the grounds are added, as a set; variables bound only in a negation stay local to it. Results become actions, including local structure reached from a result's value, a substate included (traced from preferences made at the results' level, with the identities joined to the linking value), and removals of superstate elements. The first condition on a state reached through a ^superstate element tests that it is a state.
Names and limits	chunk[xN]*rule*impasse*tDC-N as 9.6.5 makes them (Section 11); max-chunks and max-dupes per decision; a duplicate of a learned rule is not added and its results are dropped, after which the existing rule fires in the superstate.
Learned rules at work	A learned rule's instantiation is found among its matches and takes over the results (i-supported while it matches, or o-supported); learning repeats from it when results belong to a still higher state (chunks named chunkx2…). Learned rules then match and fire like any other.

Element	Formalization
Settings	chunk always|never|only|except (also flagged and unflagged) with force-learn and dont-learn; max-chunks; max-dupes; automatically-create-singletons; chunk singleton; bottom-only; add-osk; multi-attributes (condition costs); $ and $$ in conditions (a marked variable is literalized in every explanation through the rule).
add-ltm-links	With chunk add-ltm-links on, each identifier of a learned rule's results that is local to the substate and linked to an LTI gets an action (link-stm-to-ltm <v> n), taken in the order the results' symbols are variablized (add_LTM_linking_actions).
Not formalized	Operator selection knowledge from a rule that matches only the top state. It needs a substate returned as a result; Soar then backtraces into the top-state rule's instantiation, which has no identities, and learns a rule with literal identifiers (Sections 11 and 12). Excluded by decision: RL templates in learned rules and the explainer.

Proved. Results with learning on change only the procedural module's records: architecture-created elements and the goal stack are unchanged, so every earlier theorem about the cycle still holds with learning on. Two lemmas about the identity structure; two about operator selection knowledge (adding a preference to it keeps the preferences already there and adds no preference that was not a candidate); and one that the elements an impasse creates for its items are never operator selections, which keeps the decision theorems valid with the impasse items now generated by the model. The learning computation itself is executable but not yet the subject of proofs (Section 14).

Tested. soar-lean --chunks-file writes the learned rules; the comparison script reads Soar's print --chunks and compares each rule up to renaming of variables, order of conditions and actions, and the placement of relational tests between variables, as well as decisions and final working memory. Rule names are compared without the last counter, which numbers the rules learned within a decision in an order that depends on Soar's internal firing order; rules written as :chunk in an agent's own source are loaded, not learned, and are left out. All 86 chunking unit-test agents in the Soar repository match in every respect, as do 17 variants of them run with bottom-only on or with automatic singletons off (tests/chunkextra), and 5 programs that learn through episodic-memory queries and retrievals or with add-ltm-links on and off (tests/chunkmem). On the repository's 13 demonstration agents, run with Soar's default settings (operator selection knowledge on), all 13 agents match in every respect over 25 decision cycles: multi-column arithmetic, five blocks-world agents (hierarchical, hierarchical with look-ahead, look-ahead, look-ahead with state evaluation, operator subgoaling), the eight puzzle, graph search, missionaries and cannibals planning, Towers of Hanoi (recursive; 22 rules, learned in chains up to ten levels deep), and three water-jug agents (hierarchy, look-ahead, ties). Run for 50 decision cycles without replaying Soar's choices, 12 of the 13 match in every respect (several halt sooner); the recursive Towers of Hanoi agent did not finish within 40 minutes in the model. Eight findings were needed to reach this, each confirmed in the 9.6.5 source and, for most, with a debug build of Soar: the reorderer's reversed relational tests lose their identity in a learned rule's own instantiation, so rules learned bottom-up through it lose those tests (Section 12); the Rete's o-support classification across matches (Section 5 and Section 12); the arguments of every right-hand-side function that Soar registers as literalizing, compute-range among them, are literalized; the comparison now tells apart operators that differ only in unnamed objects (Section 5); Soar pushes each ground onto the front of its list (add_to_grounds), so a learned rule's conditions come in the reverse of the order the grounds were found and a constraint goes on the last

ground found that tests its identity; a rule is stored, printed and checked for duplicates as Soar's reorderer leaves it, with the tests the reorderer takes out put back on the first condition, in the new order, that binds their variables, and with the state test on the first condition on each state; rule repair gives an ungrounded identifier the variable of the condition or action that left it ungrounded, not another variable the rule has for the same identifier; and bottom-up learning continues through every state the results pass on their way up (the model had stopped after six levels; Towers of Hanoi learns chains ten deep). With rules ten levels deep the matcher needed an index on identifiers to finish in reasonable time; it is proved to return what the original matcher returns (Section 4).

What the comparison changed in the core. Eight behaviors of the core model were found wrong with the chunking tests and corrected from Soar 9.6.5: with learning on, the dependencies of an o-supported result enter the goal dependency set of the learned rule's instantiation, which supports the result, and none when that rule was learned for the top state (an operator applied in a substate by sending a command to the top state's output link had been making its substate regenerate); the production types :o-support and :i-support (manual §3.3.3) are honored; (interrupt) stops at the end of the current phase rather than at once as halt does; a substate is removed as soon as the preferences no longer give its impasse, within the phase (decision_consistent_with_current_preferences), not at the next decision; relational tests compare symbolic constants alphabetically and count a symbol of another type as less; an unquoted letter followed by digits is read as an upper-case constant; a numeric-indifferent preference may have a computed value; and when every remaining condition tests a variable not yet bound, the relational test is moved to the condition that binds it, as Soar's reorderer does.

Phase 2 changes to the core. The second phase found eleven more: the ^item-count, ^non-numeric and ^non-numeric-count elements are remade whenever an impasse's items change; a state no-change substate is removed as soon as its superstate's operator preferences change, even when no candidate results (Soar's consistency check finds no impasse where the decision had found a no-change); the application phase runs to quiescence, and deselecting a substate's operator stops o-supported firings only at that level and below; an operator proposal made by an o-supported rule, or through an attribute variable bound to operator, takes part in the decision; a halt takes effect after the whole elaboration cycle, whose inner loop still fires the assertions (not the retractions) pending at lower states; the o-supported preferences a substate instantiation makes for its results stay while that substate exists, besides those of the learned rule; succeeded, failed and success halt the agent, as Soar 9.6.5 does outside its unit-test harness; compute-range is provided; multi-attributes declarations are used in condition ordering; dot paths in smem --add make a new identifier at every step; and a learned rule is matched only against the elements it was learned from when its instantiation is made.

CMC mapping (decided). Learned rules are kept, like reinforcement learning's values, in procedural module state and matched together with production memory, so the CMC's procedural learning step is the identity. Decided October 29: keep this as is; CMC.Proc will be revised in one major iteration once the functionality is complete (Section 13).

Places where the implementation is ad hoc, and better ways. These parts reproduce Soar 9.6.5's behavior on the tests but are emulations or approximations rather than models of the mechanism; they are the first candidates for rework.

Where	What is ad hoc, and a better way
Instantiation keys	Explanation records are identified by strings (x{cycle}{i|o|c|m}{position}) and an instantiation carries the string of its record. Checked November 1: the strings are opaque (nothing parses them back; ordering uses the pass

Where	What is ad hoc, and a better way
	and sequence numbers), so they are identifiers only. A structured key and a single instantiation structure shared by preference memory and chunking would be tidier but change nothing the model computes.
Memory-system instantiations	Done October 31: an instantiation records what made it (InstKind: rule, justification, learned rule, or a memory system), and the architectural instantiation that holds semantic- and episodic-memory results is recognized by that kind rather than by the name "memory". Better still: impasse items as a kind of their own.
Results (St.resultSet)	Formed as Soar does since October 31 (get_results_for_instantiation, add_results_if_needed): a set of preferences, those of the instantiation for identifiers of higher states and, transitively, every preference of the local identifiers reached from their values and referents, each replaced by its clone at the results' level; the justification or learned rule then supports clones of them. What remains ad hoc: the learning computation still receives them as elements, operator preferences and removals (St.resultParts), and a clone is found by its content rather than by Soar's clone links. Better: give the preferences to the learning computation, and link clones.
Support classification (St.reteSupport)	Done October 31: as in Soar, each match is classified once, when the model of the Rete adds it, against the production's matches present at that moment, in the Rete's order. What remains: rules with long-term-identifier tests, matched by the model's own matcher, are classified against all their current matches whenever the match set is computed.
Reversed relational tests (learnFrom)	Which relational tests Soar's reorderer reversed is found by comparing the learned rule's conditions before and after Soar's order is applied, and those tests are left out of the rule's own instantiation record. Better: give each test its own identity, as Soar does, and reproduce the reorderer's update of it.
Retracted instantiations	An instantiation that leaves the match set is kept for backtracing while its state exists or while any of its preferences remain. Soar keeps it by reference counts from preferences and explanations. Better: reference counting, or keep it while any preference or explanation record refers to it.
The state keyword	The parser writes state as a ^type state pattern marked as a goal test, and chunking filters such patterns out by their shape. Better: a goal test on the identifier test itself.
Condition order	Two orders exist: the matcher's own (joinOrder, which also moves tests when stuck) and Soar's reorderer (soarOrder), used only to place tests in explanation records. Using Soar's order for matching was tried in the core rework and made Towers of Hanoi time out: it is chosen for a Rete network, and the model's matcher joins lists. Soar breaks ties by the numbers of the constants' symbols; since October 31 the model numbers them in the order the kernel makes them: its predefined symbols, the names of its right-hand-side functions, the source file token by token, then the constants made during the run (St.symIx, St.noteSyms). Better: an incremental matcher for which Soar's order is efficient, with a proof that reordering preserves matches, and symbol creation order recorded.

Where	What is ad hoc, and a better way
Indexed matching (matchCondsIx)	The matcher finds the elements on an identifier through a hash table built from working memory each time the match set is computed. The indexed matcher and its lookup are proved equal to the plain matcher (matchCondsIx_eq, WmIndex.onId_eq_onIdTable); what is ad hoc is rebuilding the table every time. Better: keep the index with working memory, or an incremental matcher.
Duplicate check (learnFrom)	Done October 31: a new rule is compared with each learned rule structurally, both in Soar's stored form (storedRule) with their variables renamed in order of appearance (Production.canonVars), instead of as printed text. Soar compares the new rule's Rete network with the existing ones. Better: compare in the model of the Rete, as Soar does.
Matching from scratch	Done October 31 for rules without long-term-identifier tests: the match set is kept between elaboration cycles and changed by the network's additions and removals (Section 5). What remains: rules with those tests are matched from scratch every cycle, and working memory, its linkage to states and the slots are recomputed from lists. Better: evaluate the tests in the network, and keep working memory and its linkage incrementally.
deep-copy	Implemented by rewriting the production for each match; the rewriting is repeated when firing, when recording and when noting the copies' levels, and the three must agree on identifier numbering. Better: make the copy once, as an effect of the firing. (The copies' levels now follow Soar's rule, never promoted, through the same level record as other identifiers.)
Special cases in action matching	actFor and prefAct special-case require preferences and ^operator through an attribute variable. Better: give every action and preference its identities as Soar does (pref->identities), so unification needs no cases.
The Rete's order (Soar/Rete.lean)	Modeled, with three limits: justifications are not added to the network (in Soar they share and split nodes like other rules; 300 programs built to expose an effect on the order of later matches, tests/justfuzz, all agree with Soar 9.6.5, so no effect is known); changes from more than one of Soar's batches of working-memory changes (the output phase's memory results and the next input) enter it together, at the next elaboration cycle (450 programs built to expose an effect, tests/batchfuzz, batchfuzz2 and batchfuzz3, agree with Soar 9.6.5; they found and fixed a different difference, the order in which semantic memory creates a retrieval's elements: the retrieved memory's augmentations first, then success and retrieved); and beyond 400,000 tokens the earlier ordering is used (matches in the order the rules were loaded, the first pass in the reverse order). Since October 31 the network is also the matcher (Section 5). Better: add justifications, and bring changes in at each of Soar's batches.
Symbol table (make-constant-symbol, zeros)	Soar's symbol table holds every constant still referenced. The model approximates it by the constants of the source file and of working memory and those make-constant-symbol has made, and keeps the first zero made by a function for the rest of the run, where Soar frees symbols no longer referenced. strlen and string of an identifier give no value, because the model's names for identifiers are not Soar's; timestamp gives a fixed time. A learned rule's instantiation re-evaluates its actions without working memory, so functions that read it give no value there. Better: a symbol

Where	What is ad hoc, and a better way
	table with reference counts, and identifier names as Soar makes them.
gp and source (Parse.expandGp, Main.expandSource)	Both are done on the text before parsing: gp by expanding bracket lists, source, pushd and popd by reading lines outside productions, so a command must be on a line of its own and Tcl is not interpreted. Better: a model of the command line that runs a file's commands in order, which would also give settings their position (a setting after some rules applies to the whole run here).
Deep negations (Cond.nccg)	Done November 1. A negation nested to any depth is a tree (NegTree), matched through the index and proved sound and complete (tree_complete, tree_clear). Chunking variablizes each level as it does one level, and is now tested against Soar: 12 programs with negations nested 1 to 4 deep (tests/chunkdeep), all matching, after the comparison script learned to read nested negations and Soar's one-pattern form (-^attribute).
Random-number generator	One generator per agent, seeded with 0 when there is no srand. Soar has one per process, shared by all agents in a kernel and seeded from the clock when there is no srand. The mask-and-reject loop is cut off after 64 tries (it ends with probability 1). A learned rule's instantiation re-evaluates its actions without the generator, so a rand-int value in a result is not reproduced in the learned rule's own instantiation. Better: the generator in the kernel state shared by agents, and actions evaluated once, with their values recorded.
Long-term-identifier tests (St.ltiBindings)	The model evaluates @, !@, @+ and @- against the links as they are whenever the match set is computed. Soar's Rete evaluates them when an element enters the network, so a link made later (by link-stm-to-ltm or a store) does not change a match already made or refused until the element is re-added; the tests here do not depend on the difference. Rules with these tests are matched by the model's own matcher, since the network does not evaluate them. Better: evaluate the tests when the element is added, as an incremental matcher would.
Learning eligibility	bottom-only is a list of goal identifiers in the procedural module's state that is never cleared (identifiers are not reused); it is kept there rather than on the goals because learning is proved to change only that state (learnResults_same). Singleton declarations use an enumerated kind since October 31 (SingletonKind), and, as in Soar, a declaration with any other type is refused. Better: clear the list when its goals are removed.
Working-memory activation (St.wmaPrefs, St.wmaTested, St.wmaNewWmes)	References are found by comparing preference memory and working memory before and after each elaboration cycle, and decay elements and o-sets are kept in lists keyed by timetag and preference number. Soar counts references where it adds a preference or an element. Better: have the steps that add preferences and elements emit their references, and keep the decay element with the element.
Order of duplicate preferences	Follows Soar: a pass's instantiations are made in the Rete's order and their preferences ordered as Soar adds them, so of two o-supported top-level preferences for one value the one Soar keeps is kept, and an element is supported by the first of its slot's acceptable preferences in Soar's order (PRec.aseq). Tested with activation on (tests/wmadup) and through chunking (the singleton tests, where two instantiations make one element

Where	What is ad hoc, and a better way
	and the learned rule follows the one that supports it).
Spreading activation (Spread, SMemDB.calcSpread, spSync)	The spreading tables and maps are association lists; the working-memory hooks are found by comparing a snapshot of working memory with the current one after each set of changes, rather than where Soar adds and removes elements; and a query's spread computation is applied to the store by the chain of stages of calcSpread (each proved to keep the stored content). Better: hooks in the working-memory phase itself and keyed maps.

11. Where the manual is wrong, silent or ambiguous

Each item was found by formalizing the manual's text and running the same programs in Soar 9.6.5. In each case the Lean model follows the implementation and records the manual's wording beside it.

Topic	Manual	Soar 9.6.5
Architecture attributes on states (§2.2, §3.3.6.2)	They "cannot be directly created, removed, or modified by rules"; user productions "can not create or remove" them.	Rules cannot remove them but can create them, e.g. ^impasse on the top state, or attributes under ^smem.result.
Constraint-failure substate (§3.4.1)	^choices is constraint-failure.	^choices is none.
^non-numeric (§3.4.1)	Described for tie impasses.	Also created, with ^non-numeric-count, for conflict and constraint-failure impasses.
Deselection during application (§2.5)	The application phase "will immediately end".	The phase ends only at quiescence: o-supported rules no longer fire for that state and the states below, but those of states above still do, and i-supported processing continues.
Arithmetic functions (§3.3.6.6)	Lists + - * / and then speaks of "the first three" and "the last two".	Four functions; + - * return integers for integer arguments, / returns a real number.
Order of firings in application (§2.5)	Not stated.	Pending i-supported activity fires before o-supported rules.
O-supported removal of an i-supported element (§3.3.6.3)	Only the reverse case is discussed.	The element is removed and stays removed while its instantiation still matches.
Justification conditions (§2.7.4)	"Working memory elements that exist in the superstate (and above)".	Traced to the level the result attaches to; intermediate substates are traced through.
Re-deciding among indifferent winners (§2.4)	Not stated.	The currently selected operator is kept.
Indifferent selection --	Order not stated.	The first candidate in Soar's candidate

Topic	Manual	Soar 9.6.5
first (§9)		list: operators proposed in earlier passes of the inner elaboration loop first, oldest first, and those proposed in one pass in the reverse of the order their rules fired, which follows the Rete network (Section 5, The Rete's order).
Duplicate rules (§3.3)	Not stated.	A rule is ignored as a duplicate of an earlier one even when only a constant on its right-hand side differs (val2*a with = 0.9 after val*a with = 0.5; Soar 9.6.5 prints Ignoring val2*a because it is a duplicate of val*a). This is same_rhs with RL's chunk-stop on (the default): when both rules have a single action, a preference with a numeric referent, the referents are not compared. The model does the same.
$ and $$ in conditions	Not described.	Accepted; they mark a variable to become a constant in a chunk and do not affect matching. The model literalizes a marked variable in every explanation through the rule.
hrl-discount default (§5.3.2, rl command)	§5.3.2 says the default is on; the rl command's parameter table says off.	Off.
When reward is collected (§5.2)	"At the beginning of every decision phase."	Only when an operator is selected in some state, and then for every state; a decision that creates an impasse collects nothing. This changes learned values for operators that fall into impasses.
Update on halt (§5.2)	Reward "is also collected when the agent is halted"; the update is not described.	Both the expected current and expected future reward are updated, with a next value of 0 (unlike state retraction, which updates only the expected current reward).
When stores are processed (§6.3.1)	"At the end of every phase of every decision cycle."	After every elaboration cycle of the proposal and application phases, not after the input, decision or output phases; a store issued in the application phase is visible within that phase.
Removing result elements (§6.1)	"Production actions should not remove augmentations of the result structure directly, as semantic memory will maintain these WMEs."	Rules can remove status elements such as ^success, and retrieved augmentations; doing so with status elements can corrupt the agent (Section 12).
Non-cue-based retrieval (§6.4.1)	"All augmentations of the long-term identifier in semantic memory will be	On a new identifier linked to the LTI, not on the identifier given in ^retrieve (the

Topic	Manual	Soar 9.6.5
	created as new WMEs" — on which identifier is not stated.	kernel source comment says the opposite).
^store-new with ^link-to-new-ltm (§6.3.2)	Store-new "will always store the given memory as an entirely new structure"; ^link-to-new-LTM yes relinks the identifier.	With ^link-to-new-ltm yes no new LTI is made: the existing LTI is overwritten, as by ^store. The attribute is link-to-new-ltm in lower case.
Well-formed commands (§6.4)	Only one kind of command, with its modifiers.	Validity depends on the order in which Soar reads the command's elements: ^query ^prohibit ^link-to-ltm is rejected as bad-cmd or accepted, depending on the order the rule creates them.
max and min math queries (§6.4.2)	Retrieve the LTI with the largest or smallest value.	If no stored LTI has the attribute, the condition is dropped and the least activated candidate is retrieved instead of failing.
Default episodic trigger (§7.2, §9.5)	Episodes are concluded by adding to the output link "(default)"; the parameter table also gives output.	The default is dc: an episode every decision cycle.
Candidate episodes (§7.3.1)	Candidates contain at least one cue leaf; the query returns "the most recent candidate episode with the greatest match score".	An episode is scored only where the set of matched cue leaves changes, walking back in time. If the positive cue matches nothing and the negative cue matches episode k, episode k − 1 is returned with score 0 — an episode with no cue element — not a more recent episode with score 0; with no change anywhere the query fails.
normalized-match-score (§7.3.4)	The match score divided by the cue size.	Divided by the number of positive cue leaves; cue-size counts negative leaves too (cue-size 4, score −1, normalized −0.333).
memory-id and present-id (§7.3.4)	Created for cue-based retrievals.	Created for every successful retrieval, including ^retrieve, ^next and ^previous.
Invalid ^retrieve (§7.3.2)	"Supplying an invalid value … will result in an error."	A value of 0 or less gives ^status bad-cmd; a positive number with no episode gives ^retrieved no-memory and no error.
add-osk default (§4.6, chunk command)	Operator selection knowledge is added only when add-osk is turned on; the parameter table gives off as the default.	On by default: chunk prints "Incorporate operator selection knowledge Yes" and the source sets it true. Look-ahead agents learn different rules with it on.
Automatic singletons (§4.7.3.2, chunk command)	Singletons are the architecture's plus those the user declares.	automatically-create-singletons, not in the manual and on by default, makes every element with a symbolic attribute a singleton, so conditions on the same

Topic	Manual	Soar 9.6.5
		element are always unified.
Learned-rule names (§9.4.1.6)	prefix*rule*impassetype*dc-n, impasse type one of Tie, Conflict, Failure, StateNoChange, OpNoChange; prefix chunk or justification; chunks from chunks begin chunk-xN.	The impasse is that of the superstate of the state where the rule fired, so a rule learned for the top state has none; every no-change impasse is OpNoChange (Section 12); justifications begin justify; chunks from chunks begin chunkxN with no dash.
max-chunks and max-dupes (§9.4.1)	"Per phase".	Per decision cycle: the counters are reset in the input phase.
Duplicate learned rules (§4.5)	Not stated what happens to the results.	The results are dropped; the existing rule then fires in the superstate and makes them again, with new identifiers.
State tests in learned rules (§4.4)	Not stated.	Only the variable for the state reached through a ^superstate element tests that it is a state; another variable bound to the same state (through ^top-state, say) does not.
Relational predicates (§3.3.5.3)	<, <=, >=, > are "numerically" less, greater and so on.	Symbolic constants compare alphabetically, and a symbol of a different type from the value compared against counts as less, so foo < 5 and 3 < foo both hold.
Case of constants (§3.1.1, §3.3.6)	Soar is case sensitive.	An unquoted letter followed by digits (k1, x2) is read as an upper-case constant (K1, X2); |k1| keeps its case.
When an impasse is resolved (§2.7)	By results "that lead to the selection of a new operator".	The substate is removed within the phase, as soon as the preferences no longer give its impasse, before any decision.
Repairing learned rules (§4.5.3.6)	EBC links the unconnected identifier "from a state that is already tested somewhere else in the rule" by "a breadth-first search, so it will find one with the shortest distance".	The search starts from the state at the identifier's level and is depth-first (the paths are kept on a stack), so the path found need not be the shortest; ^superstate links for states in between are added separately.
compute-range (§3.3.6.6)	Returns the distance "rounded to the nearest integer".	Truncated toward zero.
When a state no-change is resolved (§2.7)	Not stated for preferences that give no candidate.	Any change to the superstate's operator preferences removes the substate at once, even a numeric-indifferent preference that leaves no candidate; the next decision makes a new substate.
halt (§3.3.6.7)	Halts the agent.	Takes effect at the end of the elaboration cycle: the assertions

Topic	Manual	Soar 9.6.5
		pending at lower states in that cycle still fire, retractions do not.
Results (§2.7, §4)	Results are elements of a superstate made in a substate; nothing is said about a result whose value is a state.	Accepted without warning: (S1 ^sub S2) becomes a result, Soar learns a rule whose action makes a new identifier where the state was, and a top-state rule can then make preferences for the substate's operators, which chunking backtraces into without identities (Section 12). Returning a state as a result should be an error.
Working-memory activation (§9.3.2.1, wma)	Activation is increased when an element is "referenced"; which events count as references is not stated, and forgetting is described as removing elements.	Every condition of every pending match counts, at the start of each preference phase (a waiting o-supported match counts again in each elaboration cycle); an acceptable preference counts for the element of its value; i-supported elements pass references to the o-supported elements they depend on; a new element starts with the mean time sum of what its rule tested. Forgetting removes the element's o-supported preferences, so an element also supported by an i-supported rule stays.
The state symbol (§3.3.5.1)	The first condition "must match against a state" and begins with the symbol state; how a state is recognized is not stated.	A goal test: the identifier must be a state of the goal stack. A retrieved episode has ^type state but does not match (state <s> ...).
Operator elaborations (§2.7.6)	Rules that augment the selected operator are given i-support.	Only when no other match of the same rule has a different selected operator; otherwise the elaboration is o-supported and persists after the operator is deselected (Section 12).

12. Findings about the Soar 9.6.5 kernel

Formalizing Soar and comparing the model with Soar 9.6.5 found 29 defects in the Soar 9.6.5 kernel, across eight of its modules. Six are severe: a crash, two reads of freed memory, a read past the end of a symbol, a NULL dereference in a library function, and memory growth that exhausts a machine. Eighteen give wrong agent behaviour, learned rules or memory contents; five affect only names, printing, a setting with no effect or leftover database rows.

Module	Defects	The defects (severe ones first)
Condition reordering, Rete and decision cycle	5	Towers of Hanoi exhausts memory (reorderer tie-break); relational tests between different types are order dependent; support of a match depends on the production's other matches and on the

Module	Defects	The defects (severe ones first)
		warnings setting; rules differing only in a numeric-indifferent value are rejected as duplicates; every no-change impasse is typed as an operator no-change
Semantic memory	9	Crash when a state and its substate query in one output phase; a removed status element leaves a pointer to a freed preference; LTI numbers from smem --add vary between runs; the base-level approximation is never used; base-incremental-threshes reads one number and toggles it; a stored identifier joins the spreading context late; the invalidation check is never reset; printing spread tests the root LTI; opening a file database with append off leaves three tables
Episodic memory	3	Recording rewrites the LTIs of earlier episodes; graph-match mapping depends on memory addresses; merge add has no effect
Working-memory activation	3	Activation reads freed preferences; spreading's edge updating reads constants as identifiers; Petrov's approximation with decay rate 1 divides 0 by 0
Reinforcement learning	3	With eligibility traces every traced rule's instantiations get the last rule's value; with GQ(λ) the stored value is not written to the rule; a rule made from a template shares variables with the previous template rule, so distinct identifiers can become one variable
Right-hand-side function library	3	select-point-closest-to-vector dereferences NULL and set-function errors carry bars in their names; sum and product read real numbers as integers; deep-copied identifiers are never promoted
Chunking	2	Reversed relational tests lose their identity in a learned rule's own instantiation; a substate returned as a result leads to learned rules with literal identifiers
Symbol table	1	The sign of a zero depends on history

Seventeen of the defects, and two departures from the manual (store-new with link-to-new-ltm overwriting the existing LTI, and a zero-score episodic query returning an episode with no cue element), were reproduced with Soar 9.6.5; the rest follow from the source. Programs that show the main ones are in bugs/. One earlier hypothesis has been settled: the Towers of Hanoi memory growth is caused by the condition reorderer, not the Rete. The reorderer breaks ties by the order in which attributes were first created, so it places all eleven ^holds conditions of the goal-test rule before the tests that constrain them; at decision 31 that rule holds 1,417,180 tokens (3^11 at its last join), and the same count predicts the 2.7 GB at decision 32 and the failure at decision 33. The faster variant has the same rule but loads it first, and moving the rule to the top of the slow agent removes the growth.

Locations and fixes. The companion document Soar 9.6.5 Kernel Defects: Locations and Recommended Fixes (doc/Soar-9.6.5-Kernel-Defects.docx) gives, for each defect and for the two departures, its exact place in the Soar source at tag releases/9.6.5 (file, line and function), the key lines, the cause, a recommended fix as a patch sketch, what the fix would change for existing agents, and how the defect was established, with a summary table by severity and a suggested order of work. The fixes have not been compiled or tested.

13. Findings for the CMC

- **The single-act picture needs the expansion in Section 3.** Soar's automatic rule firing is not a deliberate act, yet it changes working memory. The expanded backbone (CMC.Proc) states this and shows which CMC properties survive.
- **Semantic and episodic memory: preferred mapping recorded, deferred (Laird, October 29, 2026).** The preferred option is to map both onto the CMC's single declarative module, as one module with two stores, rather than as two modules (which would amend the CMC's commitment to one long-term declarative memory). It needs four changes to CMC.Proc: declarative steps at named points inside the cycle (semantic stores after each elaboration cycle; retrievals, episodic recording and episodic retrieval at the end of the output phase, or after the decision phase with phase selection); one buffer per state for each store; weakening the protection of result areas only for the declarative result buffers, recorded as a departure, with the property that procedural memory changes the stores only through commands; and two forms of declarative learning, automatic (episodic recording) and deliberate (semantic storage). The estimated effort is half a day to a day; it has not been done. Episodic memory already meets the read-only retrieval requirement strictly (retrieval changes nothing stored, recording never changes the past, both proved); semantic retrieval changes only activations and spreading's edge weights, which the CMC allows as metadata. Until then both stores are held in procedural module state with their structures in the region.
- **Constraint failure has no CMC counterpart.** It is mapped to conflict for now; whether the CMC should distinguish it is open.
- **The status of the selected-operator augmentation is open.** It is architecture-created and cannot be removed by rules, but it was not classed as procedural process data; it is placed in the region.
- **Learning inside the cycle.** Soar's reinforcement learning and chunking change procedural knowledge during the decision, using procedural module state, whereas the CMC's learning steps are separate and see only procedural memory and working memory. Decided for now: learned values stay in procedural module state. Chunking is formalized the same way (decided October 29): learned rules are kept in procedural module state and matched with production memory, so the CMC's procedural learning step is the identity. CMC.Proc will be revised in one major iteration now that the functionality is complete.

14. Next steps

1. More agents from the Soar test suites and from CIC projects, within the supported subset.
2. The remaining proofs (Section 2): the learning computation (chunking phase 4), and that negations and Soar's reorderer keep matches when relational tests move.
3. When wanted: map semantic and episodic memory onto the CMC's declarative module as one module with two stores (Section 13; half a day to a day).
4. The recursive Towers of Hanoi demonstration agent over more than 25 decision cycles (too slow in the model at 50), and more agents with learning on.
5. The ad hoc places listed at the end of Section 10, in particular forming results as preferences.
6. Evaluate long-term-identifier tests in the network, so that every rule is matched by it (Section 5). Justifications in the Rete (300 probe programs) and Soar's batches of working-memory changes (450 probe programs) were investigated on November 1 and showed no effect on the order of matches; modeling them remains optional.

7. Material for developers connecting the Soar source to the specification: the map from each Lean definition to the Soar 9.6.5 function it models (doc/Soar-Lean-map.xlsx) has been reviewed line by line for six areas (126 of 607 definitions: 71 agree, 44 differ, 11 are data types). The right-hand-side differences were run against Soar, and six were fixed; most of those in the parser and the production language concern malformed or rarely used input and are not yet confirmed by runs. The other areas remain to be reviewed. CLAUDE.md, doc/HARNESS.md and doc/DIFFERENCES.md give the build and comparison steps, versions, run times and the diagnosis of each remaining difference.
8. Report the manual issues and the kernel defects to the Soar maintainers, with the programs that show them.
9. Optional: a review of the C++ kernel for bugs and performance (Appendix D), and a Rust implementation (Appendix C), now on hold.

Appendix A. Files

File	Lines	Contents
CMC/Base.lean, CMC/Meta.lean	—	The existing CMC specifications, unchanged in content (edits for Lean 4.35.0 below the table)
CMC/Proc.lean	300	Expanded procedural module; 9 theorems
Soar/Syntax.lean	510	Working memory, production language, printer to Soar source
Soar/Match.lean	1,060	Matcher (with deep negations and long-term-identifier tests), the indexed matcher the program runs, condition ordering, and Soar's reorderer (soarOrder); 29 theorems
Soar/MatchComplete.lean	1,020	Completeness of the matcher for positive conditions, negations and negated conjunctions, and independence of the order of patterns; 49 theorems
Soar/JoinOrder.lean	450	The join order returns the rule's own conditions, reordered; when a moved relational test keeps its meaning; 30 theorems
Soar/JoinOrderSem.lean	1,040	When moved relational tests compare values of one kind, the join order keeps the matches (joinOrder_posSat, joinOrder_matches, joinOrder_matches_rev)
Soar/ReachProps.lean	380	reach is correct: the identifiers reachable from the roots (mem_reach_iff)
Soar/SyncProps.lean	450	Bringing the Rete up to date twice is the same as once (sync_idem); 35 theorems
Soar/SupportProps.lean	110	With no matches pending, Rete-based support is the o-support rule (reteSupport_nil); 5 theorems
CMC/Util.lean	20	Point update of a function (upd) and its lemmas, shared by CMC/Base.lean and CMC/Meta.lean
Soar/Seriality.lean	300	At most one selected operator per state over whole runs; 26 theorems
Soar/Prefs.lean	410	Preference resolution and operator selection knowledge; 20 theorems
Soar/Rhs.lean	680	Right-hand-side values and Soar 9.6.5's function library, and the right-hand-side state (random numbers, make-constant-symbol)
Soar/Rand.lean, Soar/Explore.lean	390	Soar's random-number generator; indifferent selection by the exploration policies; 10 theorems
Soar/Cycle.lean	4,290	Preference memory, timetags in Soar's order of creation, the elaboration loops, decision cycle, input, impasses, substates, results, goal dependency set, reinforcement learning with its variants and templates, semantic- and episodic-memory commands and results, working-memory activation (references, initial boost, forgetting) and spreading activation's working-memory hooks, explanation records and learned rules; 8 theorems
Soar/Rete.lean	970	The order of Soar 9.6.5's Rete: the network (node sharing, MP

File	Lines	Contents
		nodes merged and split, negative and NCC nodes), its memories newest first, left and right unlinking, the steps at which matches are added and removed
Soar/Chunk.lean	940	Chunking: identities, dependency analysis, singletons, constraints, operator selection knowledge, variablization, rule formation, repair and naming
Soar/WMA.lean	240	Working-memory activation: decay elements, their arithmetic, forgetting estimates (references and forgetting are in Soar/Cycle.lean)
Soar/SMem.lean	1,160	The semantic store: storage, queries, base-level and spreading activation, smem --add, --remove and --query
Soar/SMemProps.lean	560	Properties of semantic memory; 68 theorems
Soar/EpMem.lean	1,130	The episode store: encoder, recording, reconstruction, interval search, graph match, commands
Soar/EpMemProps.lean	370	Properties of episodic memory; 40 theorems
Soar/Props.lean	910	Properties of the cycle, working-memory activation, reinforcement learning and chunking; 106 theorems
Soar/Conform.lean	290	Soar as a CMC instance; 8 theorems, among them that a CMC cycle is a Soar cycle
Soar/Parse.lean, Main.lean	1,490	Parser for Soar source; the soar-lean program
tools/	—	Generators of test programs and the comparison harness (tools/chunkcmp.py compares learned rules)

Move to Lean 4.35.0 (October 30, 2026). The specification was written on Lean 4.15.0 and moved to 4.35.0 with no change of meaning; the regression against Soar 9.6.5 gives the same results on both. The changes: meta is now a Lean keyword, so the field named meta (in CMC.Base, CMC.Meta and CMC.Proc) and local variables of that name are written «meta», which is the same name; given is reserved in definition headers, so one argument was renamed (St.install); string operations that now return slices are converted back to strings (the parser, keyRank, runCmds); List.enum, which Lean dropped, is defined in Soar/Syntax.lean as before; indexOf is now idxOf, String.mk is String.ofList, and if_pos, if_true and if_false are ite_eq_left, ite_true and ite_false; and twelve proof steps were adjusted to library lemmas whose arguments became implicit (List.mem_cons_self, List.mem_map_of_mem) or that simp now states differently (rlTune_preserves_conditions, match_sound). Every theorem still depends only on Lean's standard axioms (propext, Classical.choice, Quot.sound).

Appendix B. User guide: running Soar programs on the Lean model

This appendix explains how to build the system, run Soar programs and rule sets on the formal model, compare runs with Soar 9.6.5, and check the proofs. Commands are for Linux or macOS; paths are relative to the project directory, cmcsoar, which the zip file unpacks into.

B.1 What is in the package

Path	Contents
CMC/, Soar/, CMC.lean, Soar.lean	The Lean specification (Appendix A)
Main.lean	The soar-lean program
lakefile.lean, lean-toolchain	Build description; the toolchain file pins Lean 4.35.0
tools/	diff.py (comparison with Soar 9.6.5), run_all.sh (every test set, with the options each needs), firecmp.py (order of rule firings), ttcmp.py (timetags and identifiers of every element), flatten.py (multi-file agents), firstdiff.py, bench.py, random-program generators
tests/	Every test set in this report, as .soar files with .n files giving the number of decision cycles
bugs/	Programs showing Soar 9.6.5 defects and departures from the manual (Sections 11 and 12)
doc/	This report and the script that builds it; the map from Lean definitions to Soar functions (Soar-Lean-map.xlsx); the kernel defects with their locations and recommended fixes (Soar-9.6.5-Kernel-Defects.docx, Section 12); the plan of remaining work (Soar-on-CMC-Lean-Remaining-Work.docx); notes on the comparison harness (HARNESS.md) and on how each remaining difference was diagnosed (DIFFERENCES.md)
CLAUDE.md, README.md, STATUS.md	Orientation for developers and agents working on the code; the layout and how to run it; the current status

B.2 Installing

Lean. Install elan, the Lean version manager, then build. elan reads lean-toolchain and installs Lean 4.35.0 on first use. No libraries beyond Lean's core are needed. The first build checks every definition and proof and takes a few minutes.

```
curl https://raw.githubusercontent.com/leanprover/elan/master/elan-init.sh -sSf | sh
source ~/.elan/env
cd cmcsoar
lake build             # checks all definitions and proofs
lake build soar-lean   # builds the program .lake/build/bin/soar-lean
```

Run lake build soar-lean again after any change to the Lean files: lake build alone does not relink the program.

Soar 9.6.5 (only for comparisons and for flattening multi-file agents). Python 3 and the Soar Python package:

```
pip install soar-sml==9.6.5
```

Rebuilding this report (optional). Node.js and the docx package: `npm install -g docx`, then `cd doc; NODE_PATH=$(npm root -g) node build_soar.js Soar-on-CMC-Lean.docx results.json`.

B.3 Running a program

```
.lake/build/bin/soar-lean FILE.soar [options]
```

Option	Effect
-d N	Run N decision cycles (default 100). Each cycle is input, proposal, decision, application and output. The run stops early at (halt) or (interrupt).
--wm	Print the final working memory
--watch	Print every rule firing and retraction, by cycle, goal level and wave (IE for i-supported, PE for o-supported), like Soar's watch 3
--trace-file F	Write the decision trace to F
--wm-file F	Write the final working memory to F
--rl	Turn reinforcement learning on (B.6)
--rl-values F	Write each RL rule's update count and value to F, in the form of Soar's print --rl
--smem-file F	Write the final semantic store to F (B.5)
--ep-file F	Write every stored episode to F (B.5)
--wma-debug F	Write each decay element's history to F, as Soar's wma --history (B.5)
--smem-debug F	Write semantic memory's activation bookkeeping to F: the clock, access records and histories, prohibits, and the spreading tables (context, likelihoods, spread values)
--chunks-file F	Write the learned rules to F in Soar syntax (B.6)
--hints F	Follow recorded choices among indifferent operators (B.7)
--rete-debug F	Write the Rete's size, any new match it did not have, every element with its timetag, the current matches with the step at which each was added, and each rule's conditions in the Rete's order
--no-rete	Match with the model's own matcher and take new matches in the order the rules were loaded instead of the Rete's (the earlier approximation, for comparison)
--check-match	On every cycle, compare the Rete's matches with the model's own matcher (matches, their number, bindings and support), and report differences with --rete-debug
--db-dir D	Resolve the relative paths of semantic- and episodic-memory database files (smem/epmem --set path, smem --backup) under D (B.5)
--settings "C1; C2"	Commands read before the file (srand, decide indifferent-selection, decide numeric-indifferent-mode)
--input F	Change the input link before each input phase as the script F says (B.9)

Example. A file count.soar:

```
# Count to 3, then halt.
sp {propose*init (state <s> ^superstate nil -^count) --> (<s> ^operator <o> + =)
(<o> ^name init)}
sp {apply*init (state <s> ^operator.name init) --> (<s> ^count 0)}
```

```
sp {propose*count (state <s> ^count {<c> < 3}) --> (<s> ^operator <o> + =) (<o>
^name count)}
sp {apply*count (state <s> ^operator.name count ^count <c>)
   --> (<s> ^count <c> - ^count (+ <c> 1)) (write |count | (+ <c> 1) (crlf))}
sp {done (state <s> ^count 3) --> (halt)}
```

Running it with `soar-lean count.soar -d 10 --wm` prints:

```
5 productions loaded
1: O: init
2: O: count
3: O: count
4: O: count
--- write output ---
count 1
count 2
count 3

System halted.
--- working memory ---
1 type state
1 superstate nil
1 io #ID2
2 input-link #ID3
2 output-link #ID4
1 epmem #ID5
...
1 count 3
```

Reading the output. The decision trace has Soar's format: N: O: name when an operator is selected, N: ==>S: (impasse) when a substate is created. Working memory is printed one element per line, identifier attribute value. Identifiers are numbers; #ID marks an identifier value; a trailing + marks an acceptable preference. The top state is 1, its io object 2, the input link 3 and the output link 4. Every state also has the ^epmem, ^smem and ^reward-link structures that Soar 9.6.5 creates. A run that reaches the limit of 100 elaboration cycles in a phase (Soar's max-elaborations) prints N: max-elaborations reached.

With `--watch`, each cycle is listed as, for example:

```
2 selected name=count
2 L1 PE
  Firing apply*count :O
2 L1 IE
  Firing propose*count
  Retracting propose*count
  Retracting apply*count
```

B.4 What a .soar file may contain

Productions. Every sp {…} block is read. Supported:

- Conditions: identifier, attribute and value tests with variables, constants (symbols, integers, floating-point numbers, |quoted| strings), the predicates < <= > >= <> <=>, disjunctions << … >>, conjunctive tests { … }, attribute paths ^a.b.c, multi-valued shorthand ^a v1 v2, structured values ^a (<x> ^b c), negated conditions -(…) and -^a v, negated conjunctions -{ … } (nested to any depth), acceptable-preference tests ^operator <o> +, the long-term-identifier tests @+, @-, @ and

!@ (Section 7), the state and impasse keywords, the production types :o-support, :i-support and :template (others such as :default and :chunk are accepted and ignored), and $ and $$ (a marked variable becomes a constant in learned rules).

- Actions: (<id> ^a v) and (<id> ^a v -); all eleven operator preferences (+ - ! ~ > < = and binary forms, numeric indifference); attribute paths and multiple values; new identifiers.
- Right-hand-side functions: + - * / div mod abs sqrt sin cos atan2 int float (both also of strings, as strtoll and strtod read them) min max round-off round-off-heading compute-heading compute-range extrapolate-x-position extrapolate-y-position haversine compute-dice-probability select-point-closest-to-vector size sum product set-count set-sum set-multiply set-min set-max set-range set-mean set-stdev rand-int rand-float string strlen trim capitalize-symbol ifeq make-constant-symbol dc timestamp accept @ concat deep-copy crlf in values (timestamp gives a fixed time, accept no value); (write …), (link-stm-to-ltm <x> n), (halt), (interrupt) (stops at the end of the phase), (cmd epmem --set trigger|force|graph-match …), (force-learn <s>) and (dont-learn <s>) as actions; log, trace, count and wait, and any function that returns a value, are accepted as actions and change nothing the model represents.

Settings read from the file. Lines outside sp blocks are ignored, except these:

Line	Effect
smem --enable, smem -e, smem --set learning on	Turn semantic memory on
smem --set activate-on-query on|off; activate-on-add on|off; activation-mode recency|frequency|base-level; base-decay d; base-update-policy stable|naive|incremental; base-incremental-threshes n; base-unused-age-offset n; base-inhibition on|off; thresh n	Semantic-memory parameters (base-incremental-threshes adds or removes one number, as in Soar)
smem --set spreading on|off; spreading-continue-probability p; spreading-baseline b; spreading-depth-limit n; spreading-limit n; spreading-loop-avoidance on|off; spreading-wma-source on|off; spreading-edge-updating on|off; spreading-edge-update-factor u	Spreading activation (Section 7). As in Soar: spreading needs activation-mode base-level, the activation mode cannot change while it is on, the parameters other than edge updating cannot change once it is on, and the depth and spreading limits not once the store is in use
smem --add { … }	Add clauses to the semantic store (variables are local to each command; @n and aliases allowed)
smem --remove {(@n ^attr v …)}; smem --query {(<c> ^attr v …)} [n]	Remove values, attributes or all augmentations of an LTI; a query from the command line (no output)
epmem --set learning on (or epmem --enable)	Turn episodic memory on
epmem --set trigger dc|output|none; epmem --set phase output|selection	When episodes are recorded
epmem --set merge none|add; epmem --set lazy-commit on|off	Accepted; no effect in Soar 9.6.5
epmem --set graph-match on|off; epmem --set graph-match-ordering undefined|dfs|mcv	Graph matching in cue-based retrieval
epmem --set exclusions X	Exclude attribute X from episodes, or include it again if already excluded (as in Soar)
epmem --set balance b	Below 1, cue literals are weighted by working-memory

Line	Effect
	activation
wma --set activation on|off; decay-rate d; decay-thresh t; petrov-approx on|off; forgetting off|naive|bsearch|on; forget-wme all|lti; fake-forgetting on|off	Working-memory activation (Section 9); as in Soar, settings other than activation are ignored while activation is on
chunk always|never|only|except (also flagged, unflagged)	When rules are learned (default never); only and except use force-learn and dont-learn
chunk add-ltm-links on|off	Learned rules link their results to the LTIs the original results were linked to
chunk max-chunks N; chunk max-dupes N; chunk automatically-create-singletons on|off	Chunking limits and singletons
chunk add-osk on|off; chunk bottom-only on|off; chunk singleton <id-type> <attr> <value-type>	Operator selection knowledge (default on), learning only at the bottom state, declared singletons
multi-attributes <attr> [n] (also production optimize-attribute)	Expected number of values, used in condition ordering

Also read: soar max-elaborations, max-goal-depth, wait-snc and keep-all-top-oprefs (and the older max-elaborations and waitsnc); rl --set learning, learning-rate, discount-rate, temporal-extension, temporal-discount, hrl-discount, learning-policy, eligibility-trace-decay-rate, eligibility-trace-tolerance, decay-mode, step-size-parameter and meta-learning-rate; and source, pushd, popd and cd, which are followed when the file is read, a sourced file being read from its own directory, as in Soar. Everything else is ignored: watch, learn, other chunk settings, other decide and soar settings, other rl settings (such as apoptosis and chunk-stop), and other commands. srand, decide indifferent-selection (the policy, --epsilon, --temperature, --auto-reduce, --reduction-policy and --reduction-rate) and decide numeric-indifferent-mode are read, also under their older names indifferent-selection and numeric-indifferent-mode; the default policy is softmax, as in Soar, and without srand the seed is 0. Reinforcement learning is turned on with --rl or with rl --set learning on in the file. The order of candidates proposed in one pass follows the order in which the pass fires its matches, taken from the model of Soar's Rete (Section 5); with --no-rete, or beyond the Rete's token limit, it may differ from Soar's (to follow Soar's choices, use --hints; B.7).

Not supported (the parser stops with a message naming the production): binary preferences on attributes other than ^operator (an error in Soar too; other preferences there are ignored with Soar's warning), and right-hand-side actions other than those listed. Not modeled, though accepted: RL templates in chunking, and the spatial-visual system. Without --input the input link stays empty.

Multi-file agents. soar-lean follows source, pushd and popd itself. An agent that loads files with Tcl or other commands can still be flattened through Soar 9.6.5, which follows its source commands and prints every production in canonical form:

```
python3 tools/flatten.py path/to/agent/load.soar flat.soar
.lake/build/bin/soar-lean flat.soar -d 200
```

flatten.py keeps only the productions. If the agent turns semantic or episodic memory on, or adds to semantic memory, copy those lines (B.4 table) into flat.soar.

Writing rule sets for the model. Any Soar 9.6 rule set within the supported subset runs as it would in Soar. The first condition of each rule must test a state. As in Soar, a rule that tests the selected operator and changes the state gives o-supported (persistent) results, and other rules give i-supported results that retract when their conditions stop matching. The run is limited only by -d, (halt) and the elaboration limit, so a task should halt when done or be run for a fixed number of cycles. Output-link commands appear in the final working memory under identifier 4.

B.5 Semantic and episodic memory

Turn either memory on with the settings in B.4; the rules use the ^smem and ^epmem command and result structures as in Soar (manual Chapters 6 and 7). Two options write the final contents of the stores, and a third shows working-memory activation:

- `--smem-file` F writes one line per augmentation, lti attribute value (an LTI value is written @n), and one line lti ACT value per LTI with its activation.
- With working-memory activation on, the working-memory file (--wm-file, and --wm) shows each element's activation after its value as Soar's print does, for example S1 count 12 [4.5]; `--wma-debug` F writes each decay element's history in the form of Soar's wma --history, and soar-lean reports any reads of a freed preference (Section 12).
- `--ep-file` F writes one line per element of each episode: episode parent attribute value. Nodes are written #n; node 0 is the top state; an identifier that stood for an LTI is written #n@lti.

```
1 0 io #1
1 0 operator* #3
1 1 input-link #4
1 0 step 0
1 3 name step
```

Stores kept in files. With smem --set database file and smem --set path F (or the same for epmem), soar-lean writes the store to F when the run ends and, with append on (the default for smem; epmem needs epmem --set append on), a later run that names the same file starts from it, as Soar does (Sections 7 and 8). Relative paths are taken from the current directory, or from D with --db-dir D. smem --backup F, after a command has opened the store, writes a copy of it to F; smem --clear empties it. The files are soar-lean's own text format, not Soar's SQLite databases.

B.6 Reinforcement learning and chunking

`soar-lean agent.soar --rl` runs with learning on (learning rate 0.3, discount 0.9, temporal extension on, hrl-discount off, Sarsa, no eligibility traces, normal decay — Soar 9.6.5's defaults); rl --set lines in the file change them (B.4). A rule is an RL rule when its only action is a numeric-indifferent preference with a constant value, as in Soar. Reward is read from each state's ^reward-link. `--rl-values` F writes one line per RL rule: name, number of updates, value.

Chunking. Put chunk always (or only or except) in the file. `soar-lean agent.soar -d 20 --chunks-file chunks.soar` runs with learning and writes every learned rule, for example:

```
sp {chunkx3*condition-match*elaborate*matching-argument-object*t4-1
   :chunk
   (state <s1> ^superstate nil)
   (<s1> ^condition <i2>)
   (<i2> ^arg <i3>)
   (<i2> ^name <c4>)
   (<i3> ^type <c4>)
```

```
-->
   (<s1> ^result <i5>)
   (<i5> ^object <i2>)
}
```

Learned rules are matched with the file's rules from the moment they are learned. Their names are Soar's; their variables are named by kind (s for states, i for other identifiers, c for constants, n for variables local to a negation). The settings are Soar 9.6.5's defaults (operator selection knowledge on, automatic singletons on), so an agent compares with Soar as it stands.

B.7 Comparing with Soar 9.6.5

tools/diff.py runs every .soar file in a directory in both Soar 9.6.5 and soar-lean and compares them. Each program X.soar needs a file X.n holding the number of decision cycles to run. Soar runs with indifferent selection --first (the same settings are passed to soar-lean with --settings), and with chunking off unless --chunk is given; both systems run the same number of cycles, and Soar finishes the last cycle's application and output phases so the two final states are comparable.

```
echo 10 > mydir/count.n
python3 tools/diff.py mydir --exe

== count MATCH (4 decisions, 14 WMEs)
```

Mode	Compares
--exe	Decision traces and final working memory (identifiers abstracted)
--replay	As --exe, but the model follows Soar's choice wherever operators are mutually indifferent (Soar's choice there depends on its matcher's internal order). Soar's choices are recorded in X.hints, which soar-lean --hints X.hints reads; each choice names the operator by its augmentations (an identifier value by its own augmentations, below that by its ^name or else its constant-valued augmentations).
--rl --explore	Runs both with learning on and epsilon-greedy exploration (epsilon 0.3, seed 42), replays Soar's exploratory choices, and also compares every RL rule's update count and value; add --noreplay to let the model make its own choices with the same seed
--smem	Also compares the contents and activations of the semantic store (LTIs up to renaming)
--epmem	Also compares every stored episode with Soar's epmem --print (nodes up to renaming); combine with --replay for agents with indifferent choices

A line == X MATCH means decisions, working memory and, where selected, the learned values or stores are identical. == X DIFF is followed by the two decision traces where they differ, the working-memory elements found only in Soar or only in the model, and for stores the differing episodes or LTIs. SOURCE-ERROR means Soar rejected the file; LEAN-ERROR means soar-lean did (usually an unsupported feature, with the reason); LEAN-TIMEOUT means the model took longer than LEAN_TIMEOUT seconds (default 600; set the environment variable to change it). python3 tools/firstdiff.py DIR [mode] reports only the first differing decision of each program. SOAR_LEAN can name a different soar-lean binary.

Two runs, for databases on disk. A program X.soar with a companion X.pre.soar (and X.pre.n) is run in two agents: X.pre.soar runs first for X.pre.n decisions in an agent that is then destroyed, leaving its store

in the file at its --set path, and X.soar then runs in a new agent that opens it. Soar's files are SQLite; soar-lean's are its own, kept in the directory .lean-db (--db-dir). The files named by --set path and --backup are deleted before the first run. tests/dbfile holds these programs.

Learned rules. `python3 tools/diff.py DIR --chunk` also compares the rules learned (Soar's print --chunks with --chunks-file), up to renaming of variables and order; add --replay for agents with indifferent choices. `ONLY=name python3 tools/diff.py DIR …` runs only the files whose names contain name. tests/chunkall holds the Soar repository's chunking unit tests and tests/chunkdemo its demonstration agents.

All test sets. `tools/run_all.sh results.txt` runs every set in tests/ with the options it needs (including the sets run again without Soar's choices replayed) and writes a line per set with the number matching, followed by the programs that differ; it takes several hours on one core, and its six groups can be run in parallel (GROUP=A to E, or W). **Random programs.** The generators write .soar and .n files into a directory: `python3 tools/gen_epfuzz.py DIR 100` (similarly gen_epfuzz2, gen_smemfuzz, gen_smemfuzz2, gen_smembl, gen_wma and gen_spread; gen_rlfuzz.py SEED N DIR; gen_fuzz.py and gen_fuzz2.py write the Lean files Soar/Fuzz.lean and Soar/Fuzz2.lean). **Where an agent departs from Soar.** `python3 tools/firecmp.py agent.soar N` prints the first rule firing, in N decisions, that differs from Soar 9.6.5's, with the firings around it; `python3 tools/ttcmp.py agent.soar N` compares every element of the final working memory with its timetag, matching identifiers as they appear (TTRANGE=lo-hi prints a range of timetags), which shows where the two first made different elements or the same elements in a different order. **Timing.** `python3 tools/bench.py N agent.soar …` times N decisions in both systems.

B.8 Checking the proofs

`lake build` checks every theorem; a failure is reported with the file and line. To check one file after editing it, `lake env lean Soar/EpMemProps.lean`. To see that a theorem uses only Lean's standard axioms, add a line such as `#print axioms Soar.EpDB.newEpisode_past` to a file that imports Soar and run it with lake env lean. The theorems are in CMC/Proc.lean, Soar/Match.lean, Soar/Prefs.lean, Soar/Explore.lean, Soar/Cycle.lean, Soar/Props.lean, Soar/SMemProps.lean, Soar/EpMemProps.lean and Soar/Conform.lean, each with a statement in its doc comment.

B.9 Input from an environment

A script. `soar-lean agent.soar -d 10 --input agent.input` changes the input link before the input phase of each decision cycle as the script says. Each line is K + ID ^attr VALUE (add the element before cycle K) or K - ID ^attr VALUE (remove it). ID is I2, the input link, or #name, an identifier the script names: + with a #name value makes a new identifier, and later lines with #name refer to it. A VALUE is #name, or a constant written as in Soar source. A # followed by a space starts a comment.

```
1 + I2 ^sensor #a
1 + #a ^name left
1 + #a ^value 3
3 - #a ^value 3
3 + #a ^value 7
5 - I2 ^sensor #a
```

Comparing with Soar. tools/diff.py gives X.input to Soar 9.6.5 through SML's input functions and to soar-lean with --input. For an environment that reacts to the agent, write X.env.py with a function step(k, output), called before the input phase of cycle k with the output link's elements as (id, attr, value) strings

and returning changes in the script's form; diff.py runs it against Soar, records the changes in X.input.rec and replays them to soar-lean (tests/input/walker.env.py is an example).

B.10 Using the model from Lean

The model can also be run inside Lean, for example to supply input-link contents from a simulated environment. Parse.parseFile reads Soar source; St.init is the initial state; St.run runs a number of cycles with an environment whose input function receives the decision count and the output-link contents and returns the input-link elements.

```
import Soar
open Soar

def src : String := "sp {propose*init (state <s> ^superstate nil -^count) --> ...}"

-- put (I3 ^tick n) on the input link every cycle
def env : Env := { input := fun dc _ => [{ id := inputLink, attr := "tick",
val := .int dc }] }

#eval match Parse.parseFile src with
  | .ok pm => (St.init.run pm env 10).log      -- the decision trace
  | .error e => [e]
```

Save this as a file in the project and run `lake env lean File.lean`. The state returned by St.run gives working memory (.wm), write output (.out), the stores (.smem, .ep) and learned values (.rlValues). Semantic and episodic memory are turned on by setting smemOn and epOn in the initial state, as Main.lean does; Main.lean also sets wma, chunkCfg, multi and symRank (the constant order used by the reorderer) from the file.

B.11 Troubleshooting

Symptom	Cause and remedy
parse error: … is not supported	The production uses a feature outside the supported subset (B.4); rewrite it or leave it out.
The trace stops early	A rule fired (halt) or (interrupt), or a state no-change impasse repeats because no operator is proposed.
N: max-elaborations reached	A rule loop: some rule keeps firing within a phase. Soar prints a warning at the same point and, like the model, ends the phase there.
DIFF only in decisions among equally preferred operators	Run diff.py with --replay.
soar-lean is slow on a large agent	The model is written for clarity; large agents take seconds to minutes (Appendix E).
Changes to Lean files have no effect	Run lake build soar-lean.

Appendix C. Converting Soar to Rust

The specification makes a rewrite of Soar in Rust a much better-defined project than it would otherwise be. It supplies exact semantics, an executable reference that can be run side by side with the new code, and the comparison harness already used against Soar 9.6.5.

Potential value

- **Memory safety.** Rust rules out the classes of error — use after free, buffer overruns, data races — that are hardest to find in a large C++ kernel.
- **A kernel that conforms by construction.** Each Rust component can be checked against the corresponding Lean definition, and the theorems (rules never change architecture structures; at most one operator selection per decision; quiescence is a truth-maintenance fixpoint) become runtime assertions or property tests.
- **Maintainability.** A modern codebase with a standard build and package system (Cargo), easier to embed in other systems and to compile to WebAssembly for browser-based teaching and demonstrations.
- **A cleaner home for new mechanisms.** Extensions specified in Lean first (for example, CMC metacognition structures) can be implemented once, in both the specification and the kernel, with the same tests.

The main risks are performance parity (the C++ kernel's Rete matcher is heavily tuned) and the long tail of features that existing agents depend on.

Phases, time and cost

Phase	Scope	Agent time	Usage cost
1. Core	The decision cycle, preferences, truth maintenance and the function library, simple matcher, .soar parser; matching 9.6.5 on the test corpus	2–4 days	Tens to a few hundred dollars
2. Performance	Incremental (Rete) matcher, incremental truth maintenance and goal dependency set; still checked against Lean and 9.6.5	1–3 weeks	A few hundred dollars
3. Parity for typical agents	The learning and memory mechanisms (reinforcement learning, chunking, semantic memory with base-level and spreading activation, episodic memory, working-memory activation), command interface and an SML-compatible API	1–3 months	A few thousand to low tens of thousands of dollars
Spatial-visual system	Separate project	—	—

Usage costs are orders of magnitude at API list rates and depend on the plan in use; current pricing should be checked before committing. The larger cost is expert time: someone who knows Soar's internals reviewing design choices, deciding behaviors where the manual and the implementation differ, and testing on CIC agents — a few hours a week through phases 2 and 3. A proof that the Rust code implements the specification, as opposed to testing it, is possible in principle (tools such as Aeneas or

Verus) but is a research-scale effort; differential testing gives most of the confidence at a fraction of the cost. The Rust work is on hold.

Appendix D. Improving the current C/C++ kernel

The tools built here apply directly to the existing kernel.

Error identification

- **Differential testing at scale.** The random-program generators and the Lean model act as an oracle: any program where 9.6.5 and the specification disagree is either a kernel bug or a manual error. The generators can be extended to deeper substates, results and justifications, the goal dependency set, and floating-point arithmetic, and run on thousands of programs.
- **Sanitizers.** Building the kernel with AddressSanitizer and UndefinedBehaviorSanitizer and running its unit tests and performance agents finds memory and undefined-behavior errors that do not show up as wrong behavior.
- **Resource pathologies.** The Towers of Hanoi memory growth (Section 12) shows that some agents drive the matcher into extreme states; systematic runs with memory limits on the test agents would find others.
- **Documentation.** The 43 manual issues in Section 11 can be resolved one by one, either by changing the manual or, where the implementation is unintended, the kernel.

Performance

- **Profiling the performance agents** in the repository to find hot spots, typically the Rete matcher, preference and slot handling, goal dependency set bookkeeping, and memory allocation.
- **Condition ordering.** The Towers of Hanoi case suggests that the kernel's condition reordering can leave a large join unconstrained; a better ordering heuristic could remove such blow-ups. Each change is checked by the same differential tests, so behavior cannot shift silently.

How much speed-up is available cannot be known until profiling is done; for typical agents the kernel is already fast (Appendix E), so the main gains are likely to be in pathological cases and in large agents.

Activity	Agent time	Usage cost
Bug-finding pass: sanitizers, scaled differential testing, report with reproducible cases	1–2 weeks	A few hundred dollars
First optimization round: profiling, measured changes, regression checks	2–4 weeks	Low thousands of dollars

As with Rust, the dominant cost is maintainer time to review and merge changes, perhaps a few hours a week.

Appendix E. Execution time: Lean model versus the C++ kernel

Each agent was run for the same number of decision cycles (40, or fewer where the agent halts) in Soar 9.6.5 and in soar-lean, on the same two-core cloud machine, with tracing off; each time is the best of three runs. Soar times are for the run command inside the kernel; Lean times exclude start-up and

parsing (shown separately). For agents with indifferent choices the Lean run followed Soar's recorded choices, so both took the same path. The Towers of Hanoi agent is excluded; the faster variant of it, which does not show the memory growth, is included.

Agent	Decisions	Soar 9.6.5 (s)	Lean (s)	Lean ÷ Soar	Lean parsing (s)
Arithmetic	40	0.0009	1.24	1,438	0.039
Blocks world	40	0.0008	0.09	121	0.010
Blocks world, look-ahead	25	0.0015	0.28	189	0.031
Blocks world, operator subgoaling	5	0.0003	0.02	86	0.010
Count test	40	0.0003	0.04	172	0.015
Eight puzzle, look-ahead	40	0.0055	0.70	127	0.031
Towers of Hanoi (fast variant)	40	0.0014	0.27	189	0.012
Water jug	40	0.0004	0.04	112	0.007
Water jug, hierarchical	40	0.0004	0.06	155	0.010

The Lean model is between about 90 and 1,400 times slower, measured on November 1 with the model of Soar's Rete as the matcher. Since the Rete became the matcher (October 31) the --no-rete option is slower, not faster (arithmetic 2.5 s, the eight puzzle 1.7 s, blocks world with look-ahead 0.7 s). The history: before the core rework the range was 140 to 280,000; the rework made the slowest agents much faster (blocks world with operator subgoaling from 114 s to 0.1 s, arithmetic from 21 s to 10 s); the index on identifiers added to the matcher halved the time of the slowest again (arithmetic from 10 s to 4.5 s); compiling with Lean 4.35.0 instead of 4.15.0 made every agent between 7 and 46 percent faster; and using the Rete as the matcher, with parsing sped up, brought arithmetic to 1.2 s, the eight puzzle to 0.7 s and parsing to under 0.05 s for every agent (it had been about a second for the largest). The Soar times vary by up to a factor of two from run to run at this scale, so the ratios are approximate. The model was written to be read and proved about, not to be fast: it recomputes which elements are linked to a state from scratch and uses lists throughout, where the C++ kernel uses indexed data structures. The cost grows with the number of rules and the size of working memory, which is why the arithmetic agent is slowest. The model is practical for the small and medium agents used in testing, and for checking behavior; it is not a substitute for the kernel. Incremental linkage would narrow the gap further without changing the specification's meaning, but would make it harder to read and to prove about.

Appendix F. Sources

Laird, J. E., et al. The Soar Cognitive Architecture Manual, version 9.6.

Laird, J. E. (2022). Introduction to the Soar Cognitive Architecture. arXiv:2205.03854.

Laird, J. E., Lebiere, C., & Rosenbloom, P. S. (2017). A Standard Model of the Mind. AI Magazine, 38(4).

Laird, J. E., Lebiere, C., Rosenbloom, P. S., & Stocco, A. Unified, Comprehensive Metacognition within the Common Model of Cognition. AGI.

Soar 9.6.5, via the soar-sml Python package (version 9.6.5); test agents from the SoarGroup/Soar repository (UnitTests/SoarTestAgents).